



learn
enough

JAVASCRIPT

TO BE DANGEROUS

Rough Cuts



MICHAEL HARTL

Learn Enough JavaScript to Be Dangerous

A Tutorial Introduction to Programming with JavaScript

Michael Hartl

 **Addison-Wesley**

Contents

Chapter 1. Hello, World!

Chapter 2. Strings

Chapter 3. Arrays

Chapter 4. Other Native Objects

Chapter 5. Functions

Table of Contents

Chapter 1. Hello, World!

- 1.1 Introduction to JavaScript
- 1.2 JS in a Web Browser
- 1.3 JS in a REPL
- 1.4 JS in a File
- 1.5 JS in a Shell Script

Chapter 2. Strings

- 2.1 String Basics
- 2.2 Concatenation and Interpolation
- 2.3 Printing
- 2.4 Properties, Booleans, and Control Flow
- 2.5 Methods
- 2.6 String Iteration

Chapter 3. Arrays

- 3.1 Splitting
- 3.2 Array Access
- 3.3 Array Slicing
- 3.4 More Array Methods
- 3.5 Array Iteration

Chapter 4. Other Native Objects

- 4.1 Math and Number
- 4.2 Dates
- 4.3 Regular Expressions
- 4.4 Plain Objects
- 4.5 Application: Unique Words

Chapter 5. Functions

- 5.1 Function Definitions
- 5.2 Functions in a File

5.3 Method Chaining

5.4 Iteration for Each

Chapter 1. Hello, World!

As the only language that can be executed inside web browsers, *JavaScript* is an essential part of every programmer's toolkit. *Learn Enough JavaScript to Be Dangerous* is designed to get you started writing practical and modern JavaScript programs as fast as possible, using the latest JavaScript technologies (including Node.js and ES6), with a focus on the real tools used every day by software developers.

Unlike most JavaScript tutorials, we'll be treating JavaScript as a *general-purpose* programming language right from the start, so our examples won't be confined to the browser. The result is a practical narrative introduction (<https://www.learnenough.com/tutorial-writing-tutorial>) to JavaScript—a perfect complement both to in-browser coding tutorials and to the voluminous but hard-to-navigate JavaScript reference material on the Web.

You won't learn everything there is to know about JavaScript—that would take thousands of pages and centuries of effort—but you will learn enough JavaScript to be *dangerous* ([Figure 1.1](#)).¹

1. Image courtesy of Kirk Fisher/Shutterstock.



Figure 1.1: JavaScript knowledge, like Rome, wasn't built in a day.

There are no programming prerequisites for *Learn Enough JavaScript to Be Dangerous*, although it certainly won't hurt if you've programmed before. What is important is that you've started developing your *technical sophistication* ([Box 1.1](#)), either on your own or using the preceding Learn Enough tutorials (<https://www.learnenough.com/courses>). These tutorials include the following, which together make a good list of prerequisites for this book:

1. *Learn Enough Command Line to Be Dangerous* (<https://www.learnenough.com/command-line>)
2. *Learn Enough Text Editor to Be Dangerous* (<https://www.learnenough.com/text-editor>)
3. *Learn Enough Git to Be Dangerous* (<https://www.learnenough.com/git>)
4. *Learn Enough HTML to Be Dangerous* (<https://www.learnenough.com/html>)

5. *Learn Enough CSS & Layout to Be Dangerous*
(<https://www.learnenough.com/cssand-layout>)

Box 1.1: Technical Sophistication

An essential aspect of using computers is the ability to figure things out and troubleshoot on your own, a skill we at Learn Enough (<https://www.learnenough.com/>) call *technical sophistication*.

Developing technical sophistication means not only following systematic tutorials like *Learn Enough Command Line to Be Dangerous*, *Learn Enough Git to Be Dangerous*, *Learn Enough HTML to Be Dangerous*, and *Learn Enough CSS & Layout to Be Dangerous*, but also knowing when it's time to break free of a structured presentation and just start Googling around for a solution.

Learn Enough JavaScript to Be Dangerous will give us ample opportunity to practice this essential technical skill.

In particular, as alluded to above, there is a wealth of JavaScript reference material on the Web, but it can be hard to use unless you already know basically what you're doing. One goal of this tutorial is to be the key that unlocks the documentation. This will include lots of pointers to my favorite JavaScript source, the Mozilla Developer Network (MDN) Web Docs (or just “MDN” for short).

Especially as the exposition gets more advanced, I'll also frequently include the exact web searches I used to figure out how to accomplish the particular task at hand. For example, how do you use JavaScript to return all elements on a page that match, say, a particular CSS class? Like this: javascript css class return all elements.

In order to learn enough JavaScript to be dangerous, we'll begin at the beginning with a series of simple “hello, world” programs using several different techniques ([Chapter 1](#)), including an introduction to *Node.js*, a fast and widely used execution environment for JavaScript programs. In line with the Learn Enough philosophy of always doing things “for real”, even as early

as [Chapter 1](#) we'll deploy a (very simple) dynamic JavaScript application to the live Web.

After mastering “hello, world”, we'll take a tour of some JavaScript *objects*, including strings ([Chapter 2](#)), arrays ([Chapter 3](#)), and other native objects ([Chapter 4](#)). Taken together, these chapters constitute a gentle introduction to *object-oriented programming* with JavaScript.

In [Chapter 5](#), we'll learn the basics of *functions*, an essential subject for virtually every programming language. We'll then apply this knowledge to an elegant and powerful style of coding called *functional programming* ([Chapter 6](#)).

Having covered the basics of built-in JavaScript objects, in [Chapter 7](#) we'll learn how to make objects of our own. In particular, we'll define an object for a *phrase*, and then develop a method for determining whether or not the phrase is a *palindrome* (the same read forward and backward).

Our initial palindrome implementation will be rather rudimentary, but we'll extend it in [Chapter 8](#) using a powerful technique called *test-driven development* (TDD). In the process, we'll learn more about testing generally, as well as how to create and publish a self-contained software package called an *NPM module* (and thereby join the large and growing ecosystem of software packages managed by *npm*, the Node Package Manager).

In [Chapter 9](#), we'll apply our new NPM module to a JavaScript web application: a site for detecting palindromes. This will give us a chance to learn about *events* and *DOM manipulation*. We'll start with the simplest possible implementation, and then add several extensions of steadily increasing sophistication, including *alerts*, *prompts*, and an example of an HTML *form*.

In [Chapter 10](#), we'll learn how to write nontrivial *shell scripts* using JavaScript, a much-neglected topic that underscores JavaScript's growing importance as a general-purpose programming language. Examples include reading from both files and URLs, with a final example showing how to manipulate a downloaded file as if it were an HTML web page.

In [Chapter 11](#), we'll apply the techniques from [Chapter 9](#) and [Chapter 10](#) to a real, industrial-grade website. In particular, we'll extend the sample application from *Learn Enough CSS & Layout to Be Dangerous* to add a functional *image*

gallery that dynamically changes images, CSS classes, and page text in response to user clicks. (We’ll be using Git to *clone* a repository directly, so you’ll be able to build and deploy the image gallery even if you haven’t completed *Learn Enough CSS & Layout to Be Dangerous*.)

In most cases, typing in code examples by hand is the most effective way to learn, but sometimes copying and pasting is more practical. To make the latter more convenient, all code listings from this book are available online at the following URL:

```
https://github.com/learnenough/learn\_enough\_javascript\_code\_listings
```

Although full-blown web development with a dynamically rendered frontend and a database back-end is beyond the scope of this book, by the end of *Learn Enough JavaScript to Be Dangerous* you’ll have a solid foundation on which to build such skills. We’ll end the tutorial with pointers to additional resources for extending your Java-Script knowledge further, as well as to further Learn Enough tutorials for full-stack web development—specifically, using *Ruby* (via *Sinatra*) and *Ruby on Rails*, for which a background in JavaScript is excellent preparation.

1.1 Introduction to JavaScript

JavaScript was originally developed by computer scientist Brendan Eich for Netscape Navigator, the first commercial web browser, under the name “LiveScript” ([Box 1.2](#)). The original use of JavaScript is still its main use—namely, “making cool things happen on web pages”, typically via manipulation of the Document Object Model (DOM) introduced in *Learn Enough CSS & Layout to Be Dangerous*. In recent years, though, JavaScript’s role has expanded significantly, and it is now often used as a back-end and general-purpose programming language as well.

Box 1.2: What’s in a Name?

*What’s in a name? that which we call a rose
By any other name would smell as sweet;*

—William Shakespeare, *Romeo and Juliet* 2.2.45–46

What we now call *JavaScript* was originally called “LiveScript” by its creators at Netscape, but at the time of its planned release there was an enormous amount of hype about *Java*, a language developed by Sun Microsystems. In an attempt to capitalize on this hype, Netscape changed LiveScript’s name to “JavaScript”—thereby causing endless confusion for developers wondering what it has to do with Java. (The answer is: nothing.)

Later on, a standardized version of JavaScript called *ECMAScript* (pronounced ECK-muh-script) was created in an effort to improve cross-browser compatibility. Technically, what most people call “JavaScript” is more properly called “ECMAScript”, with JavaScript being only ECMAScript’s most common implementation, but in this tutorial we follow the common convention of using “JavaScript” to refer to the language in general. The main exception to this rule is our occasional use of contracted names like “ES6”, which refers to the sixth edition of ECMAScript (a particularly large and important update, adding many useful features to the ECMAScript/JavaScript standard).

Finally, it’s worth noting that the misspelling “Javascript”, with a lowercase “s”, is extremely common, to the point of being borderline acceptable, even in relatively formal contexts. I frankly find the “Javascript” spelling to be more appealing than the rather pedantic official version, but it is technically wrong, so in this tutorial we’ll stick with “JavaScript”, and be technically correct.

In order to give you the best broad-range introduction to programming with JavaScript, *Learn Enough JavaScript to Be Dangerous* uses four main methods:

1. Front-end JavaScript programs running in the user’s browser
2. An interactive prompt with a Node.js Read-Evaluate-Print Loop (REPL)
3. Standalone JavaScript files (including the Node Package Manager)
4. Shell scripts (as introduced (https://www.learnenough.com/text-editor-tutorial/advanced_text_editing#sec-writing_an_executable_script) in *Learn*

Enough Text Editor to Be Dangerous)

We'll begin our study of JavaScript with four variations on the time-honored theme of a “hello, world” program, a tradition that dates back to the early days of the C programming language. The main purpose of “hello, world” is to confirm that our system is correctly configured to execute a simple program that prints the string **hello, world!** (or some close variant) to the screen. By design, the program is simple, allowing us to focus on the challenge of getting the program to run in the first place.

Since the original and still most common application of JavaScript is to write programs that execute on the Web, we'll start by writing (and deploying!) a program to display a greeting in a web browser. We'll then write a series of three programs using the JavaScript execution system Node.js: first in the Node REPL, then in a JavaScript library file called **hello.js**, and finally in an executable shell script called **hello**.

Throughout what follows, I'll assume that you have access to a Unix-compatible system like macOS, Linux, or the Cloud9 IDE (https://www.learnenough.com/dev-environment-tutorial#sec-cloud_ide), as described in the free tutorial *Learn Enough Dev Environment to Be Dangerous* (<https://www.learnenough.com/dev-environment>). If you don't have access to such a system, it's recommended that you follow *Learn Enough Dev Environment to Be Dangerous* before proceeding. (If you use the cloud IDE, I recommend creating a development environment (https://www.learnenough.com/dev-environment-tutorial#fig-cloud9_page_aws) called **javascript-tutorial**.)

Note for Mac users: Although it shouldn't matter in *Learn Enough JavaScript to Be Dangerous*, it is recommended that you use the Bourne-again shell (Bash) rather than the default Z shell to complete this tutorial. To switch your shell to Bash, run **chsh -s /bin/bash** at the command line, enter your password, and restart your terminal program. Any resulting alert messages are safe to ignore. See the Learn Enough blog post “Using Z Shell on Macs with the Learn Enough Tutorials” (<https://news.learnenough.com/mac-os-bash-zshell>) for more information.

1.2 JS in a Web Browser

Even though JavaScript is increasingly used as a general-purpose programming language, it still thrives in its native habitat of the web browser. Accordingly, our first “hello, world” program involves displaying a notification, or *alert*, created by JavaScript code on a web page.

We’ll begin by making a directory for this tutorial using **mkdir -p** (which creates intermediate directories as necessary),² along with an HTML index file using the **touch** command:³

2. If you’re using the cloud IDE recommended in *Learn Enough Dev Environment to Be Dangerous*, I suggest replacing the home directory `~` with the directory `~/environment`, though the tutorial should work the same either way.

3. You can find coverage of Unix commands like these in *Learn Enough Command Line to Be Dangerous*.

```
$ mkdir -p ~/repos/js_tutorial
$ cd ~/repos/js_tutorial
$ touch index.html
```

Next, we’ll follow the practice introduced (https://www.learnenough.com/gittutorial/getting_started#sec-initializing_the_repo) in *Learn Enough Git to Be Dangerous* and put our project under version control with Git:

```
$ git init
$ git add -A
$ git commit -m "Initialize repository"
```

At this point, we’re ready to make our first edit. We’ll start in familiar territory by adding a simple HTML skeleton (without JavaScript) to our index page, as shown in [Listing 1.1](#). The result appears in [Figure 1.2](#).

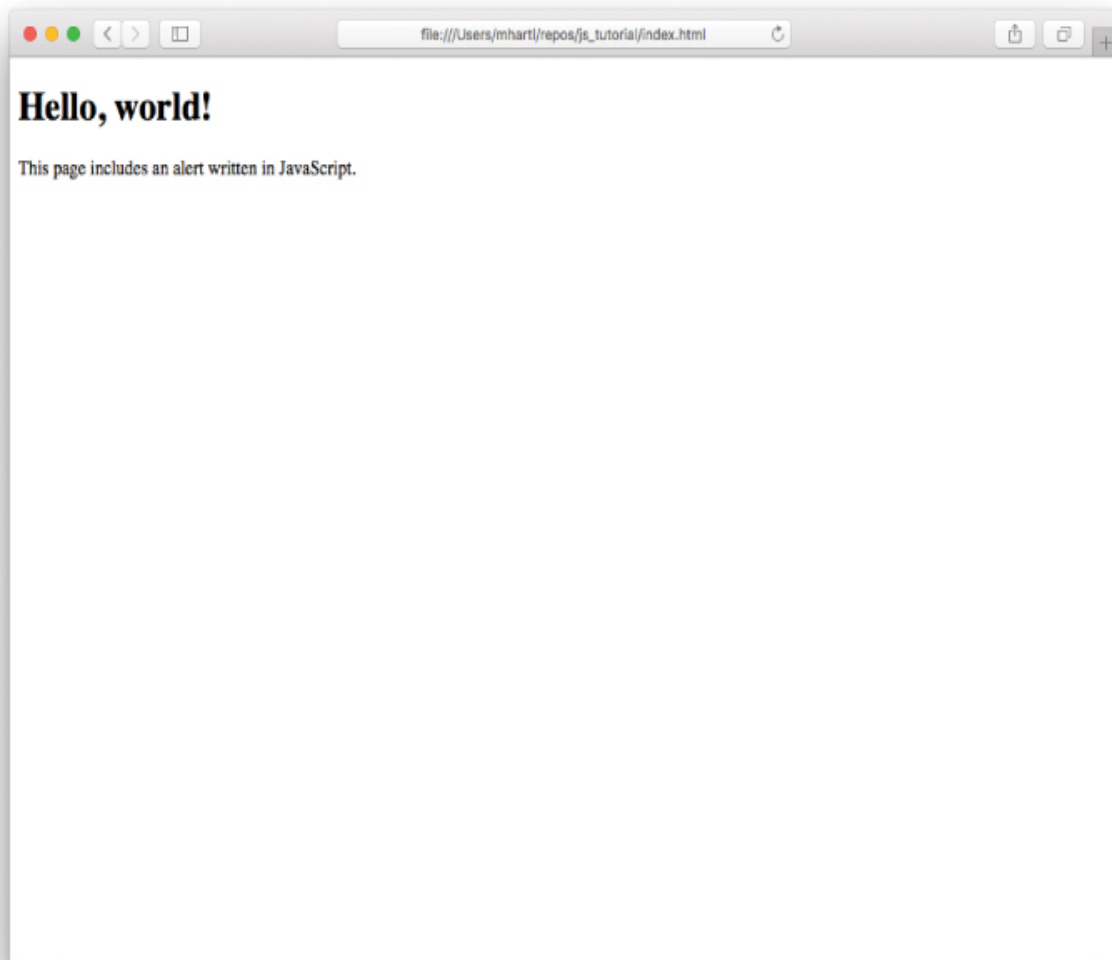


Figure 1.2: Our initial static index page.

Listing 1.1: An HTML skeleton.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
  </head>
  <body>
    <h1>Hello, world!</h1>
    <p>This page includes an alert written in JavaScript.</p>
  </body>
</html>
```

This page’s paragraph is a little lie, because we haven’t yet added any JavaScript. Let’s change that by putting in a **script** tag containing a single command:

```
<script>
  alert("hello, world!");
</script>
```

Here we’ve used **alert**, which is a JavaScript *function*, a piece of code that takes in arguments and performs some task with them. As shown in [Figure 1.3](#), the anatomy of a JavaScript function call is the function’s name, an open parenthesis, zero or more arguments, a closing parenthesis, and a semicolon to end the line. (We’ll learn more about functions, including how to define our own, in [Chapter 5](#).)

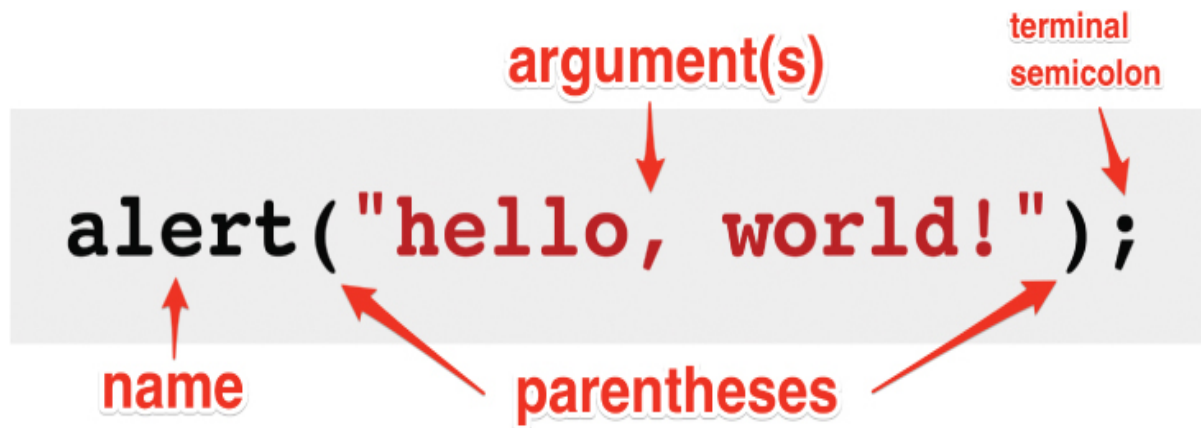


Figure 1.3: The anatomy of a JavaScript function call.

In this case, **alert** takes in a *string* ([Chapter 2](#)) and displays it as an alert in the browser. To see this in action, let’s add the **alert** code to our index page, as shown in [Listing 1.2](#). Technically, we could place the **script** tag anywhere on our page, but it’s conventional to place it in the **head** of the document (especially when including external JavaScript files, as we’ll see in [Section 5.2](#)).

Listing 1.2: “Hello, world!” in JavaScript.
index.html

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Learn Enough JavaScript</title>
  <meta charset="utf-8">
  <script>
    alert("hello, world!");
  </script>
</head>
<body>
  <h1>Hello, world!</h1>
  <p>This page includes an alert written in JavaScript.</p>
</body>
</html>
```

Upon refreshing the page, our browser now displays a friendly greeting ([Figure 1.4](#)).

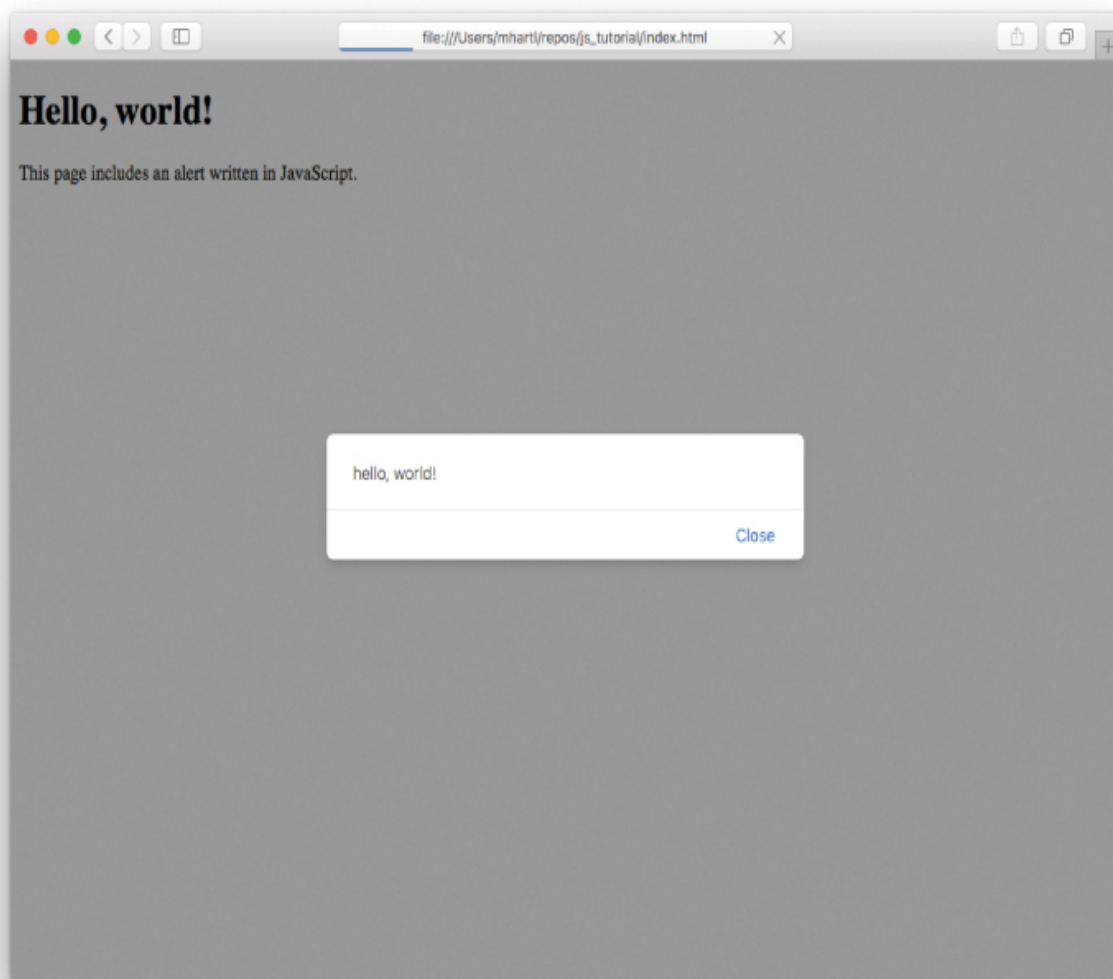


Figure 1.4: The result of a “hello, world!” alert.

1.2.1 Deployment

As a final step, let's deploy our incredibly fancy JavaScript app to the live Web. Our technique is the same one covered in *Learn Enough Git to Be Dangerous*, *Learn Enough HTML to Be Dangerous*, and *Learn Enough CSS & Layout to Be Dangerous*, namely, a free site hosted at GitHub Pages.

Deploying at even this early stage is a powerful proof-of-concept—all kidding aside about our “incredibly fancy” app, we really are deploying a live website, which was an enormously difficult step only a few years ago, and yet now we can do it in seconds.

First, let's commit the changes made in [Listing 1.2](#):

```
$ git commit -am "Add a JavaScript 'hello, world'"
```

The next step is to create a new remote repository at GitHub, as shown in [Figure 1.5](#). (If any of these steps are unfamiliar, consult *Learn Enough Git to Be Dangerous* for details.)

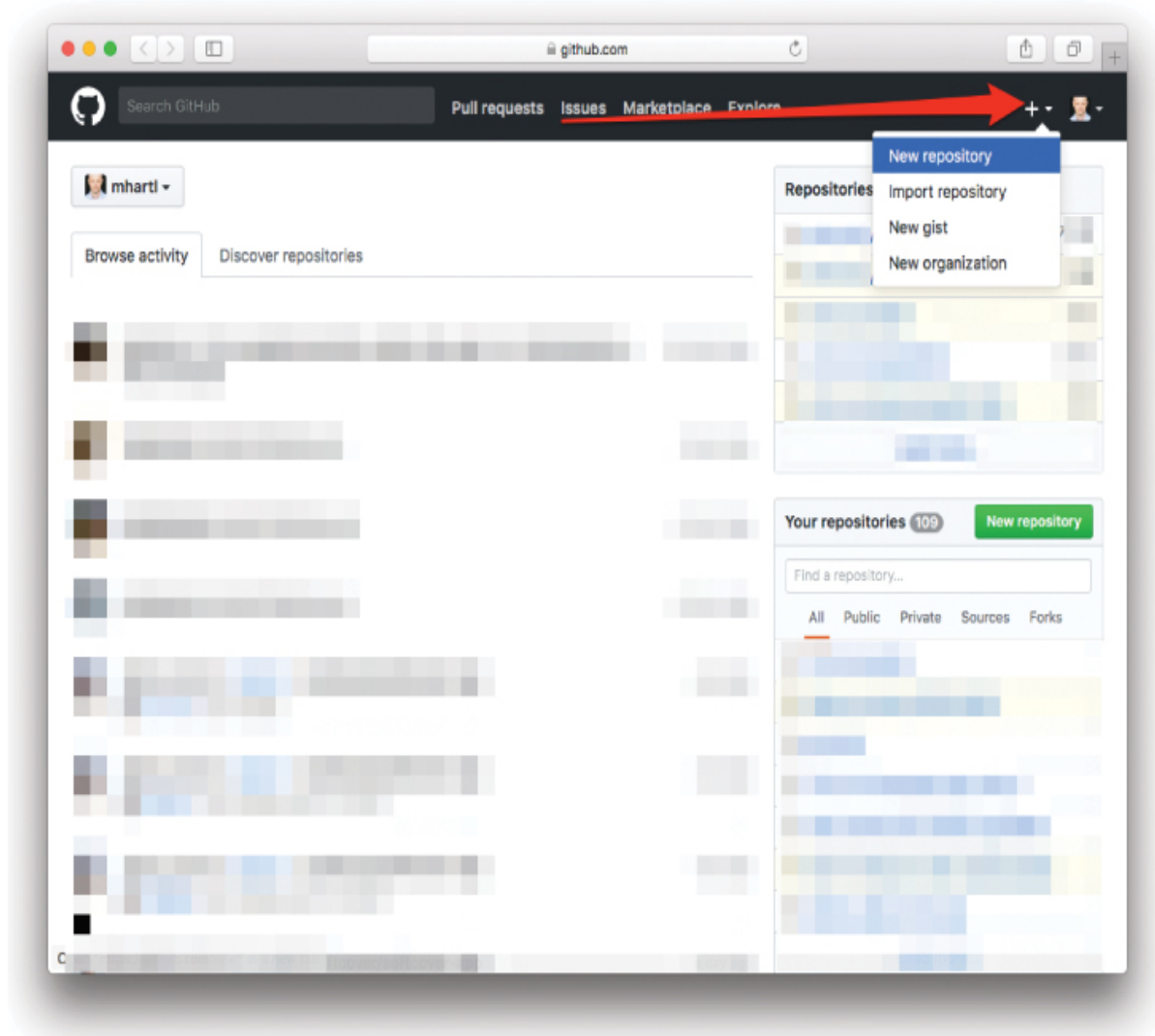


Figure 1.5: Creating a new repository at GitHub.

Next, configure your local system with the remote repository and push it up (taking care to fill in **<username>** with your GitHub username and using a GitHub personal access token when prompted for a password) and then push it up:

```
$ git remote add origin https://github.com/<username>/js_tutorial.git  
$ git push -u origin main
```

Because videos are relatively hard to update, the screencasts that accompany this book use **master**, which was the default branch name for the first 15+ years of Git's existence, but the text has been updated to use **main**, which is

the current preferred default. See the Learn Enough blog post “Default Git Branch Name with Learn Enough and the Rails Tutorial” (<https://news.learnenough.com/default-git-branch-name-with-learn-enough-and-the-rails-tutorial>) for more information.

To complete the deployment, all we need to do is edit the Settings (Figure 1.6) and configure our site to be served off the **main** branch by GitHub Pages, as shown in Figure 1.7 and Figure 1.8.

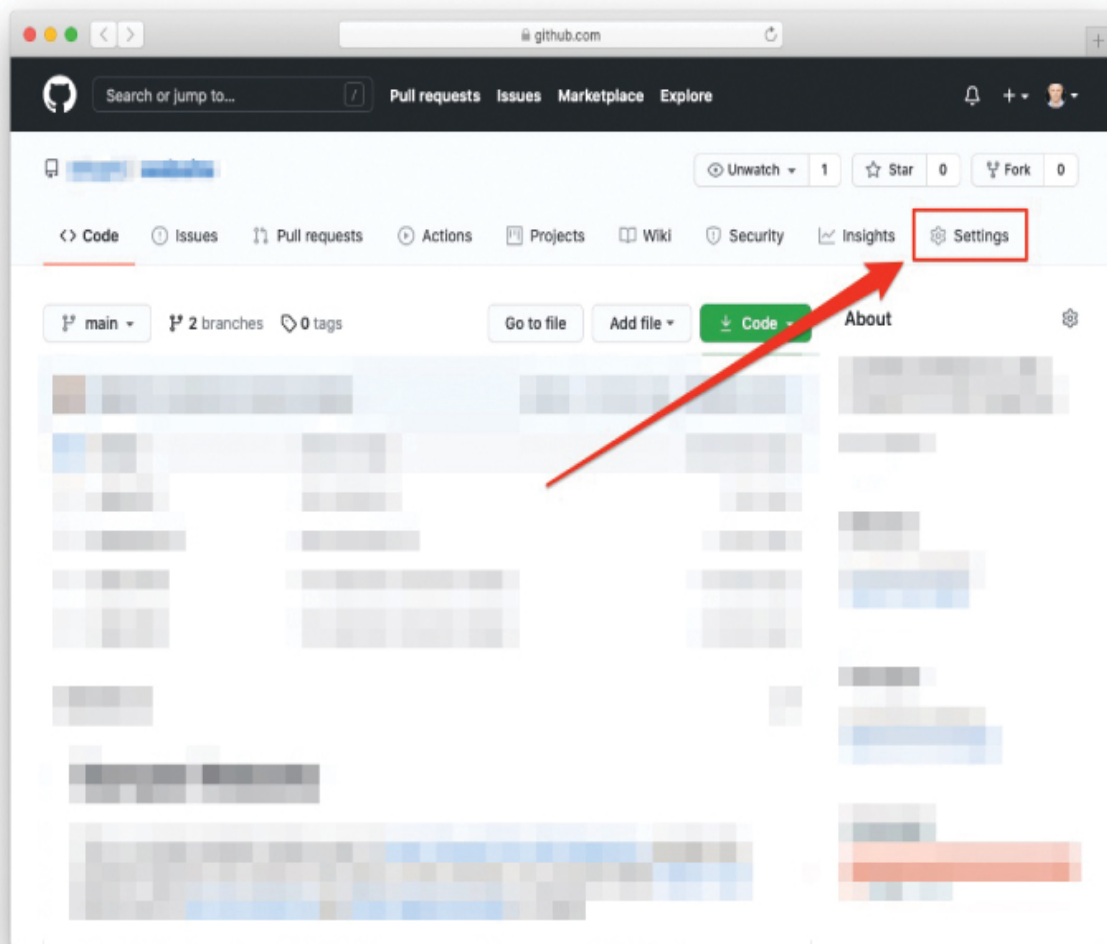


Figure 1.6: Editing the settings for a GitHub repository.

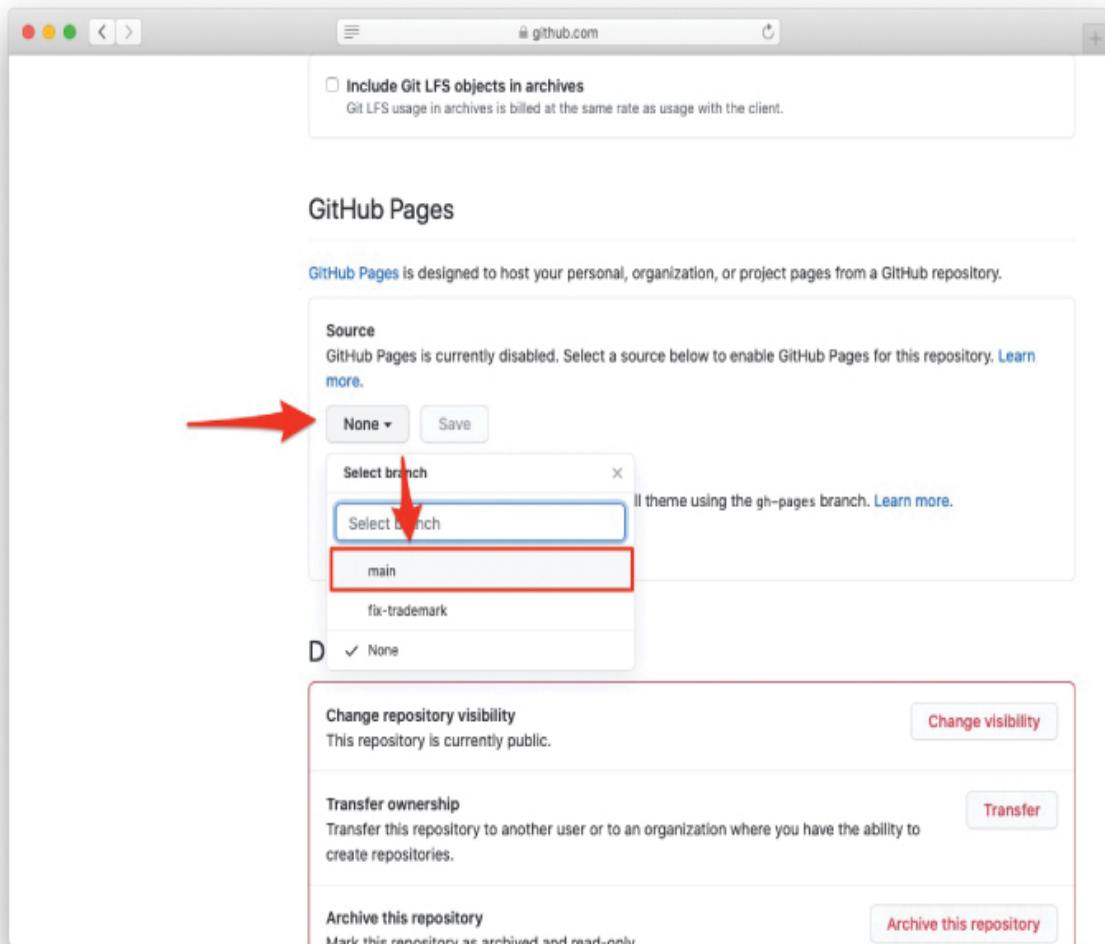


Figure 1.7: Serving our website from the **main** branch.

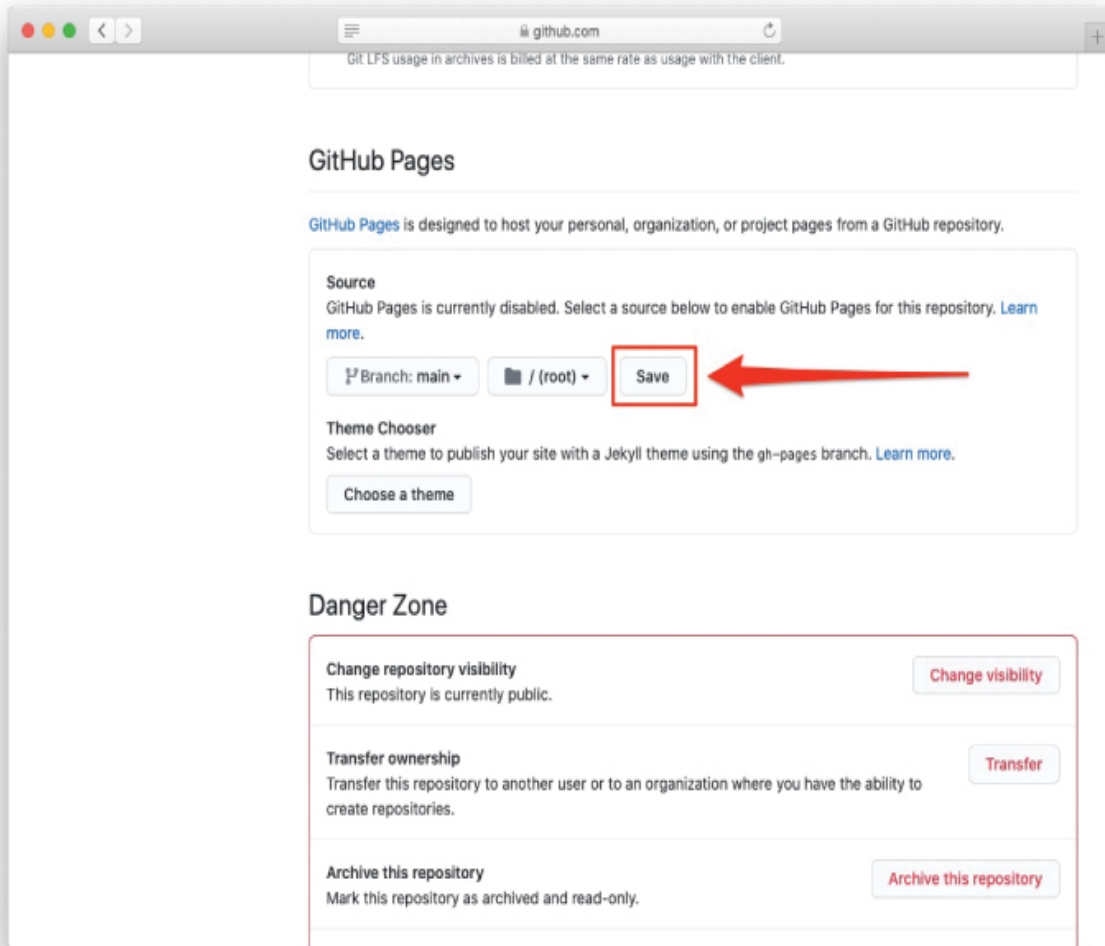


Figure 1.8: Saving the new GitHub Pages settings.

With that, you can now visit your site at the following URL:⁴

4. To learn how to host a GitHub Pages site using a custom domain, see the free tutorial *Learn Enough Custom Domains to Be Dangerous* (<https://www.learnenough.com/custom-domains>).

```
https://<username>.github.io/js_tutorial
```

The result should be the same “hello, world!” greeting seen in [Figure 1.4](#), except now on the live Web ([Figure 1.9](#)). “It’s alive!” ([Figure 1.10](#)).⁵

5. Image courtesy of Niday Picture Library/Alamy Stock Photo.

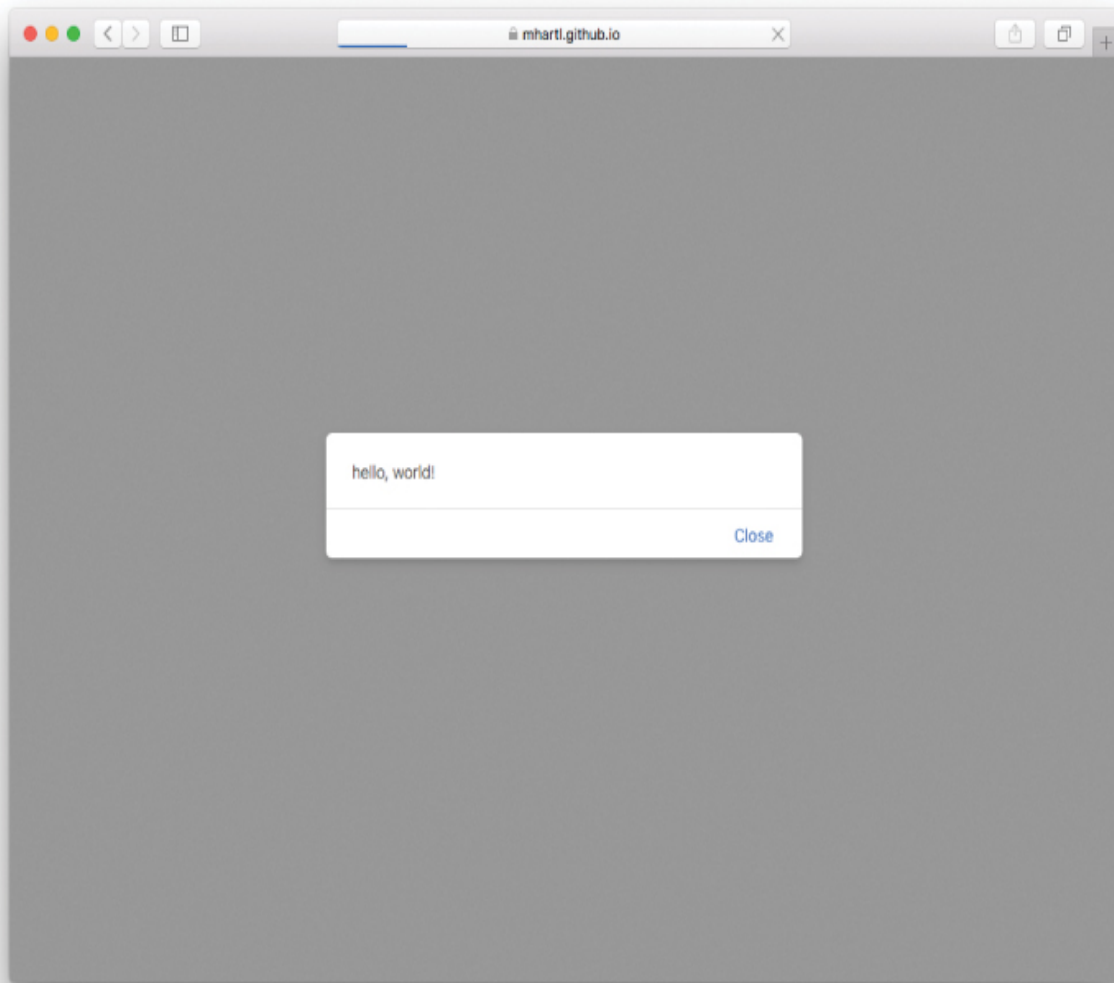


Figure 1.9: A JavaScript “hello, world!” page on the live Web.



Figure 1.10: Bringing a site to life is easier than it used to be.

1.2.2 Exercises

1. What happens if you put a second alert after the first one?

1.3 JS in a REPL

Our next two closely related examples of a “hello, world” program involve a Read-Eval-Print Loop, or *REPL* (pronounced “repple”). A REPL is a program that **reads** input, **evaluates** it, **prints** out the result (if any), and then **loops** back to the read step. Most modern programming languages provide a REPL, and JavaScript is no exception. In fact, as hinted above, it actually provides two.

1.3.1 Browser Console

Our first example of a REPL is the browser *console*, which is available in most modern browsers as part of the standard suite of developer tools. Whether these tools are available by default depends on the browser you use; they’re included automatically in Google Chrome, for example, but in Safari they have to be installed. Use your technical sophistication ([Box 1.1](#)) to figure out the setup for your browser of choice.

The developer tools can typically be accessed by right-clicking (or Ctrl-clicking) in your browser window and selecting Inspect Element to open the web inspector ([Figure 1.11](#)). The result should look something like [Figure 1.12](#).

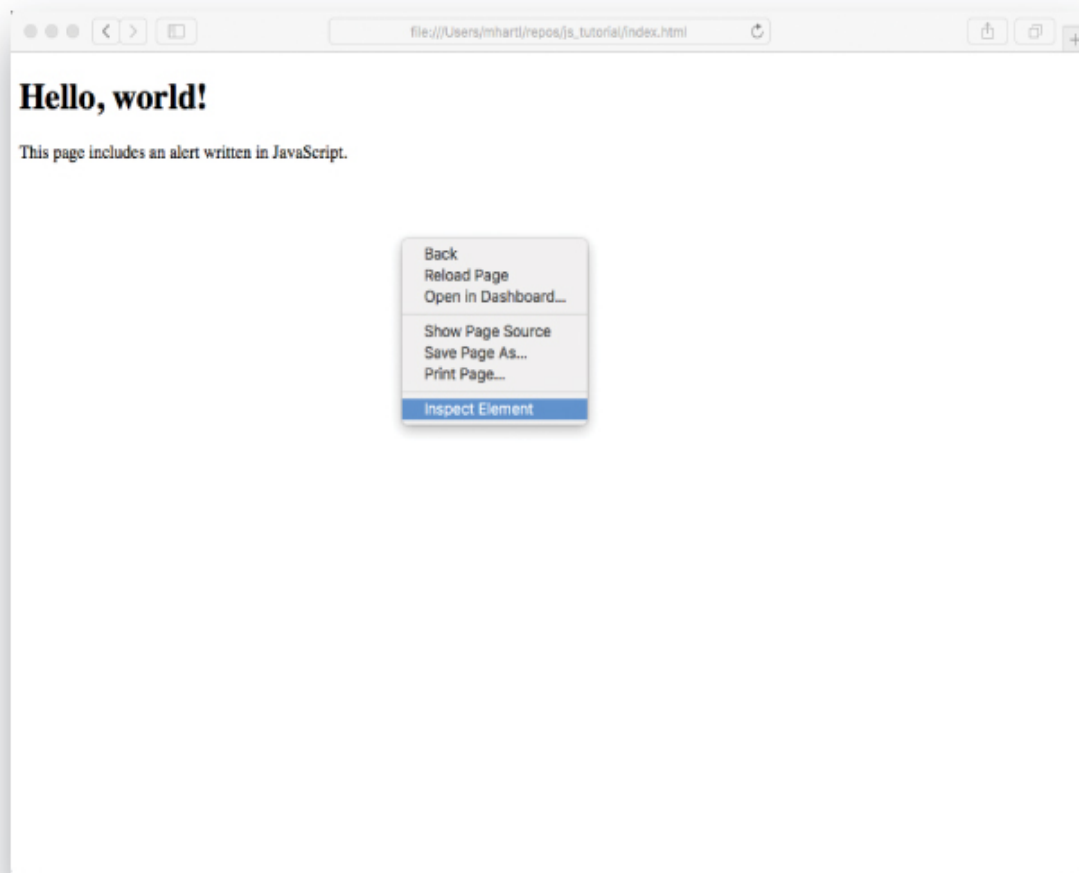


Figure 1.11: Activating the developer tools via Inspect Element.

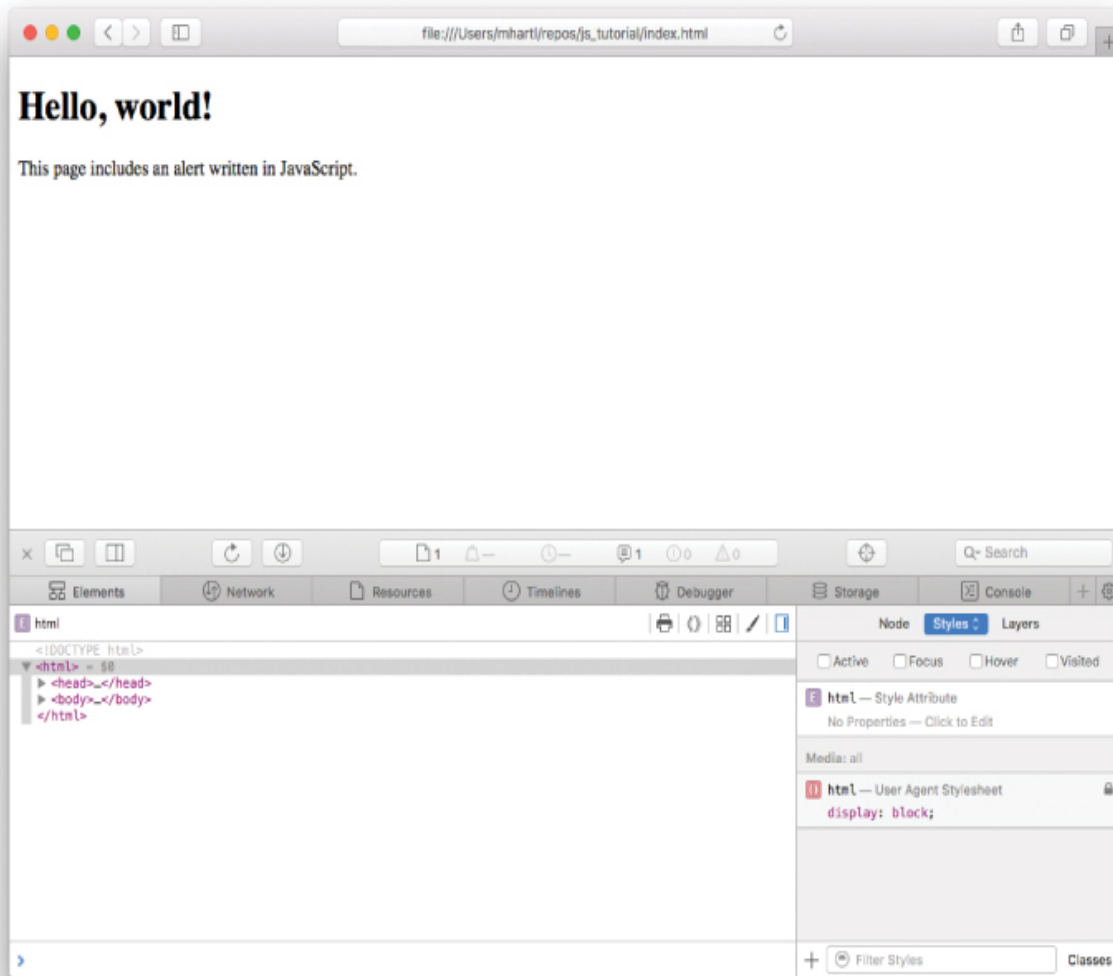


Figure 1.12: The browser developer tools.

At this point, we're ready to access the console by clicking on the corresponding tab in the developer tools, as shown in [Figure 1.13](#). As we'll see in [Section 5.2](#), the console is a valuable debugging tool, as it has access to the full DOM and other aspects of our application's environment, as well as displaying any warnings or errors that might affect our application. In particular, note that [Figure 1.13](#) shows a warning (regarding a missing **favicon.ico** file); knowing when you can and can't safely ignore such warnings is a hallmark of technical sophistication. (In this case, it's safe to ignore. In addition, your setup may or may not show the exact same error. Is that a problem?)

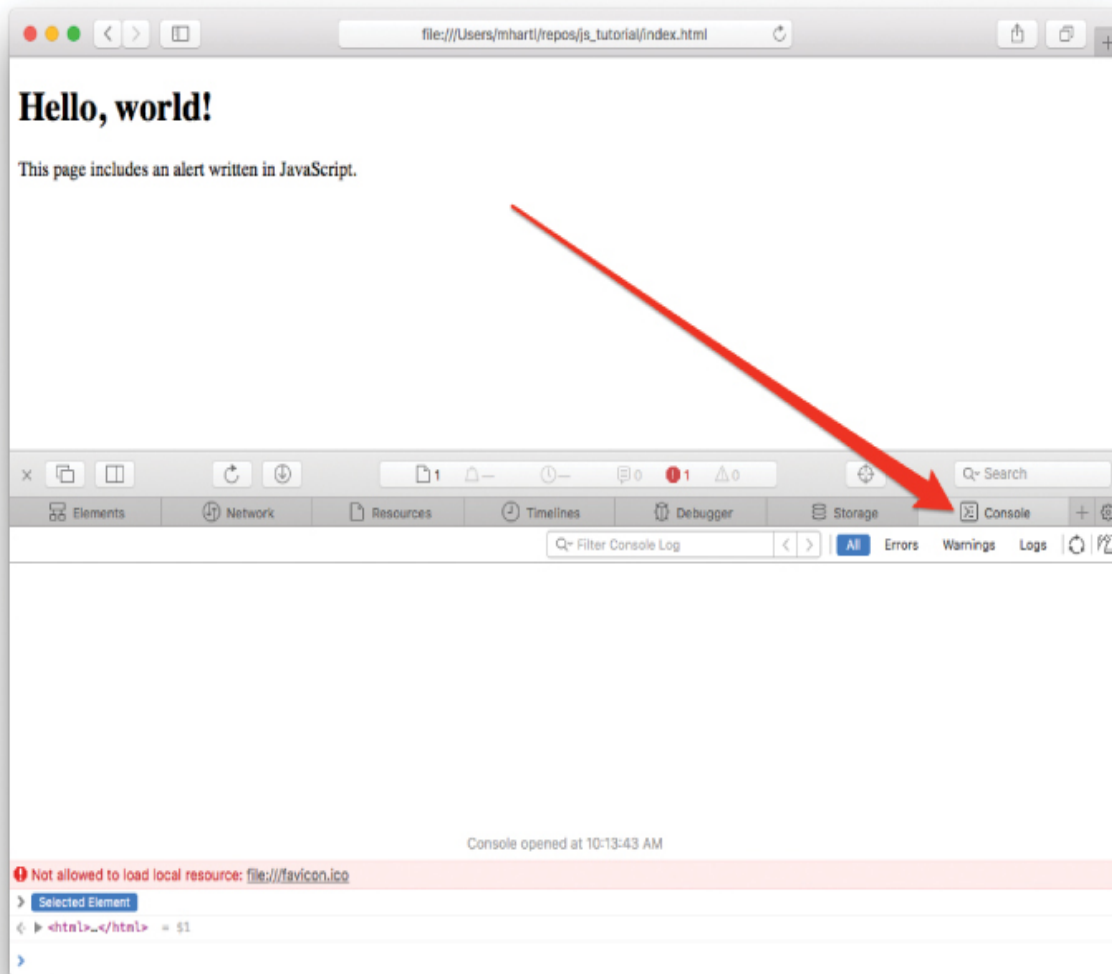


Figure 1.13: The interactive JavaScript console.

We're finally ready to write our "hello, world" program using the console REPL. Our method is to use **console**, which is a JavaScript *object* that represents the console and its associated data, functions, etc. In particular, the **console** object has a function called **log**, which prints out ("logs") its argument to the screen. We can access it using a "dot" notation that has become standard across a wide variety of *object-oriented* languages, as seen in [Listing 1.3](#).

Listing 1.3: A "hello, world" command in the console.

```
> console.log("hello, world!");
```

In the context of objects, a function like **log** called using the dot notation is often called a *method*.

At this point, you should type the **console.log** command into your browser console, noting that the **>** in [Listing 1.3](#) represents the console prompt itself, and shouldn't be typed literally. The result should resemble [Figure 1.14](#). (We'll explain the meaning of **undefined** in [Section 2.3](#).)



```
> console.log("hello, world!")
hello, world!
undefined
> |
```

Figure 1.14: Printing out “hello, world!” in the browser console.

Alert readers (no pun intended) might have noticed that the command in [Listing 1.3](#) includes a terminating semicolon ([Figure 1.3](#)), whereas the command shown in [Figure 1.14](#) doesn't. This discrepancy is included in order to show that the two commands work the same, and it is common to omit the semicolon when using an interactive console. For consistency, we'll generally include the semicolon throughout the rest of this tutorial (even in consoles), but it's good to be aware of both conventions in case you see something different in other people's code.

1.3.2 Node Prompt

Every web browser in the known Universe can execute JavaScript programs, but part of treating JavaScript as a general-purpose programming language means running it at the command line as well. This means installing and using a command-line program capable of evaluating JavaScript programs, and nowadays the most popular choice is abundantly clear: Node.js (usually pronounced “node jay-ess”, and often called “Node” for short).

It's possible that Node.js is already installed on your system. The easiest way to check is to use the **which** command (as described

(https://www.learnenough.com/command-line-tutorial/inspecting_files#sec-downloading_a_file) in *Learn Enough Command Line to Be Dangerous*:

```
$ which node
/usr/local/bin/node
```

If the path to a **node** executable is displayed, you're good to go.

If Node isn't present on your system, you should install it at this point. If you're using a Macintosh with Homebrew (<https://www.learnenough.com/dev-environment-tutorial#sec-homebrew>) installed, you can run

```
$ brew install node
```

to get the latest version. If you already have it installed, run

```
$ brew upgrade node
```

instead.

Otherwise, go to the Node.js website (<https://nodejs.org/en/>) and follow the download and installation instructions for your system.

Once it's installed, running the Node.js REPL is easy—just run the **node** command at the command line, as seen in [Listing 1.4](#).

Listing 1.4: Bringing up the Node prompt at the command line.

```
$ node
>
```

As with the browser console, **>** represents the Node prompt, and like the console it allows us to run commands interactively. (For simplicity, we'll sometimes use "console" to refer either to the browser console or to the Node REPL.) In particular, to replicate the "hello, world" program from [Listing 1.3](#), we can simply type the same command at the Node prompt, as follows:

```
> console.log("hello, world!");
hello, world!
```

(Depending on your system, you might see **undefined** appear as well; we'll discuss this detail in [Section 2.3](#).)

That's it! In both the browser console and Node prompt, we can print “hello, world!” with this single command:

```
> console.log("hello, world!");
```

1.3.3 Exercises

1. What happens if you run an **alert** in the browser console? What about in the Node console?

1.4 JS in a File

As convenient as it is to be able to explore JavaScript interactively, most Real Programming™ takes place in text files created with a text editor. In this section, we'll show how to create and execute a JavaScript file with the same “hello, world” program we've discussed in the previous two sections. The result will be a simplified prototype of the reusable JavaScript files we'll start learning about in [Section 5.2](#).

We'll start by creating a JavaScript file (with a **.js** file extension) for our **hello** program:

```
$ touch hello.js
```

Next, using our favorite text editor, we'll fill the file with the contents shown in [Listing 1.5](#). Note that the code is exactly the same as in [Listing 1.3](#) and subsequent examples, with the difference being that in a JavaScript file there's no command prompt **>**.

Listing 1.5: A “hello, world” program in a JavaScript file.

hello.js

```
console.log("hello, world!");
```

At this point, we're ready to execute our program using the same **node** command we used in [Listing 1.4](#) to bring up the Node prompt. The only difference is that this time we include an argument with the name of our file:

```
$ node hello.js  
hello, world!
```

As before, the result is to print “hello, world!”, this time to the terminal screen. (Inside the program, the return value of **console.log** is **undefined** as before, but it's not displayed since, unlike with interactive prompts, return values aren't displayed by command-line programs.)

Although this example is simple, it's a huge step forward, as we're now in the position to write JavaScript programs much longer than could comfortably fit in an interactive console or Node session.

1.4.1 Exercises

1. What happens if you give **console.log** two arguments, as in [Listing 1.6](#)?

Listing 1.6: Using two arguments.

hello.js

```
console.log("hello, world!", "how's it going?");
```

1.5 JS in a Shell Script

Although the code in [Section 1.4](#) is perfectly functional, when writing a program to be executed in the command line shell (https://www.learnenough.com/command-line-tutorial/basics#sec-man_pages) it's often better to use an *executable script* of the sort discussed in *Learn Enough Text Editor to Be Dangerous*. Now that JavaScript can be used so effectively outside the browser, it has joined more traditional “scripting languages” like Perl, Python, and Ruby as an excellent choice for writing such shell scripts.

Let's see how to make an executable script using Node. We'll start by creating a file called **hello**:

```
$ touch hello
```

Note that we *didn't* include the **.js** extension—this is because the filename itself is the user interface, and there's no reason to expose the implementation language to the user. Indeed, there's a reason not to: By using the name **hello**, we give ourselves the option to rewrite our script in a different language down the line, without changing the command our program's users have to type. (Not that it matters in this simple case, but the principle should be clear. We'll see a more realistic example in Section 10.3.)

There are two steps to writing a working script. The first is to use the same command we've seen before ([Listing 1.5](#)), preceded by a “shebang” line telling our system to use **node** to execute the script.

The exact shebang line is system-dependent; you can find the proper executable path for your system by running the **which** command:

```
$ which node
/usr/local/bin/node
```

Using this command for the shebang line in the **hello** file gives the shell script shown in [Listing 1.7](#).

Listing 1.7: A “hello, world” shell script.

hello

```
#!/usr/local/bin/node

console.log("hello, world!");
```

We could execute this file directly using the **node** command as in [Section 1.4](#), but a true shell script should be executable without the use of an auxiliary program. (That's what the shebang line is for.) Instead, we'll follow the second of the two steps mentioned above and make the file itself executable using the **chmod** (“change mode”) command combined with **+x** (“plus executable”):

```
$ chmod +x hello
```

At this point, the file should be executable, and we can execute it by preceding the command with `./`, which tells our system to look in the current directory (dot = `.`) for the executable file. (Putting the **hello** script on the PATH (https://www.learnenough.com/text-editor-tutorial/advanced_text_editing#code-export_path), so that it can be called from any directory, is left as an exercise.) The result looks like this:

```
$ ./hello  
hello, world!
```

Success! We've now written a working JavaScript shell script suitable for extension and elaboration. As mentioned briefly above, we'll see an example of a real-life utility script in Section 10.3.

Throughout the rest of this tutorial, we'll mainly use the Node REPL for initial investigations, but the eventual goal will almost always be to create a file (either pure code or HTML) containing JavaScript.

1.5.1 Exercises

1. By moving the file or changing your system's configuration, add the **hello** script to your environment's PATH. (You may find the steps in *Learn Enough Text Editor to Be Dangerous* helpful.) Confirm that you can run **hello** without prepending `./` to the command name.

Chapter 2. Strings

Strings are probably the most important data structure on the Web, since web pages ultimately consist of strings of characters sent from the server to the browser, and many other kinds of programs also require string manipulation. As a result, strings make an excellent place to start our JavaScript programming journey.

2.1 String Basics

Strings are made up of sequences of characters in a particular order. We’ve already seen several examples in the context of our “hello, world” programs in [Chapter 1](#). Let’s see what happens if we type a string by itself (without **console.log**) into a Node session:

```
$ node  
> "hello, world!"  
'hello, world!'
```

A sequence of characters typed literally is called a *string literal*, which we’ve created here using the double quote character `"`. The REPL prints the result of evaluating the line, which in the case of a string literal is just the string itself.

There’s one detail you might have noticed above: We entered the string using double quotes, but the REPL returned it using single quotes. This detail is system-dependent (for example, the console in browsers like Chrome and Safari uses double quotes for string return values), so you shouldn’t be concerned if your system differs. But this small discrepancy gives us an opportunity to learn about the difference between single and double quotes in JavaScript.

Unlike many other languages, JavaScript uses double and single quotes interchangeably for almost all practical purposes. The main exception is that apostrophes have to be *escaped out* with a backslash when included inside single-quoted strings:

```
> "It's not easy being green"  
'It\'s not easy being green'
```

Here the output includes a backslash in front of the apostrophe in “It’s”. If we were to type the same string without escaping the apostrophe, the REPL would think that the string ended after “It”, leading to a syntax error:

```
> 'It's not easy being green'  
'It\'s not easy being green'  
> 'It's not easy being green'  
'It's not easy being green'  
  ^  
SyntaxError: Unexpected identifier
```

What’s happening here is that JavaScript sees a bare letter **s** after the string **'It'**. Since there’s no identifier called **s**, the REPL raises an error ([Figure 2.1](#)).¹ (We’ll have more to say about identifiers in [Section 2.2](#) ([Box 2.2](#)).)

1. Image courtesy of LorraineHudgins/Shutterstock.



Figure 2.1: Sometimes it’s not easy dealing with syntax errors.

Similarly, inside double-quoted strings, literal double quotes have to be escaped out:

```
> "Let's write a \"hello, world\" program!"  
'Let\'s write a "hello, world" program!'
```

As you might have guessed, the return value shows how no escaping of double quotes is necessary inside a single-quoted string.

A particularly important string is one with no content, consisting simply of two quotes. This is known as an *empty string* (or sometimes *the empty string*):

```
> ""  
''
```

We'll have more to say about the empty string in [Section 2.4.2](#) and [Section 3.1](#).

2.1.1 Exercises

1. JavaScript supports common special characters such as tabs (`\t`) and newlines (`\n`). Show that both of these special characters work with both single- and double-quoted strings. What are their effects?

2.2 Concatenation and Interpolation

Two of the most important string operations are *concatenation* (joining strings together) and *interpolation* (putting variable content into strings).

Whether we use single- or double-quoted strings, we can concatenate (join) them with the `+` operator:²

2. This use of `+` for string concatenation is common in programming languages, but in one respect it's an unfortunate choice, because addition is the canonical commutative operation in mathematics: $a + b = b + a$. (In contrast, multiplication is in some cases non-commutative; for example, when multiplying matrices it's often the case that $AB \neq BA$.) In the case of string concatenation, though, `+` is most definitely *not* a commutative operation, since, e.g., `"foo" + "bar"` is `"foobar"`, whereas `"bar" + "foo"` is `"barfoo"`. Partially for this reason, some languages (such as PHP) use a different symbol for concatenation, such as a dot `·` (yielding `"foo" · "bar"`).

```
$ node  
> "foo" + "bar";           // String concatenation
```

```
'foobar'
```

Here the result of evaluating **"foo"** plus **"bar"** is the string **"foobar"**. (The meaning of the odd names “foo” and “bar” is discussed (https://www.learnenough.com/command-line-tutorial/manipulating_files#aside-foo_bar) in *Learn Enough Command Line to Be Dangerous* (<https://www.learnenough.com/command-line>).) Note also that the concatenation example includes a descriptive JavaScript *comment* ([Box 2.1](#)), which you wouldn’t ordinarily include in a REPL session, but will sometimes be added in this tutorial for clarity.

Box 2.1: A Comment About Comments

JavaScript *comments* start with two slash characters `//` and extend to the end of the line. Comments are ignored when JavaScript is executed, but they are useful for human readers (including, often, the original author!). In the code

```
// Prints a greeting to the console.  
console.log("hello, world!"); // The command itself
```

the first line is a comment indicating the purpose of the subsequent line, whereas the second line contains both some code and a comment describing the purpose of the line.

Sometimes you’ll want to add comments to several lines at a time (which is particularly useful for “commenting out” multiple lines of code when debugging ([Box 5.1](#))). Any good text editor (<https://www.learnenough.com/text-editor>) will allow you to select multiple lines and comment or uncomment them all simultaneously, yielding things like this:

```
// console.log("foobar");  
// console.log("racecar");  
// console.log("Racecar");
```

The details vary from editor to editor, so use your technical sophistication ([Box 1.1](#)) to figure out the command for your editor of choice.

JavaScript also supports multiline comments enclosed in `/* . . . */`, like this:

```
/* console.log("foobar");  
   console.log("racecar");  
   console.log("Racecar"); */
```

Because of the ease with which modern text editors can apply single-line comments to multiple lines, in practice I find that I rarely use the `/* . . . */` syntax.

You wouldn't ordinarily include comments in console sessions, but for instructional purposes I'll sometimes include comments in what follows, like this:

```
$ node  
> 17 + 42    // Integer addition  
59
```

If you follow along by typing or copying-and-pasting commands into your own console, you can omit the comments if you like; the console will ignore them in any case.

Let's take another look at string concatenation in the context of *variables*, which you can think of as named boxes that contain some value (as mentioned in *Learn Enough CSS & Layout to Be Dangerous* (<https://www.learnenough.com/css-and-layout>) and discussed further in [Box 2.2](#)).

Box 2.2: Variables and Identifiers

If you've never programmed a computer before, you may be unfamiliar with the term *variable*, which is an essential idea in computer science. You can think of a variable as a named box that can hold different (or "variable") content.

As a concrete analogy, consider the labeled boxes that many elementary schools provide for students to store clothing, books, backpacks, etc. ([Figure 2.2](#)). The variable is the location of the box, the label for the box is the

variable name (also called an *identifier*), and the content of the box is the variable value.



Figure 2.2: A concrete analogue of computer variables.

In practice, these different definitions are frequently conflated, and “variable” is often used for any of the three concepts (location, label, or value).

We can create variables for a first name and a last name using the JavaScript command **let**, as shown in [Listing 2.1](#).

Listing 2.1: Using **let** to assign variables.

```
> let firstName = "Michael";  
> let lastName  = "Hartl";
```

Here **let** associates the identifier **firstName** with the string **"Michael"** and the identifier **lastName** with the string **"Hartl"**.

The identifiers **firstName** and **lastName** in [Listing 2.1](#) are written in so-called CamelCase (named for the resemblance of the capital letters to humps of a camel ([Figure 2.3](#))),³ which is a common naming convention for JavaScript variables. Variable names conventionally start with a lowercase character, whereas *object prototypes* like **String** (Chapter 7) start with a capital letter.

3. Image courtesy of Utsav Academy and Art Studio. Pearson India Education Services Pvt. Ltd.

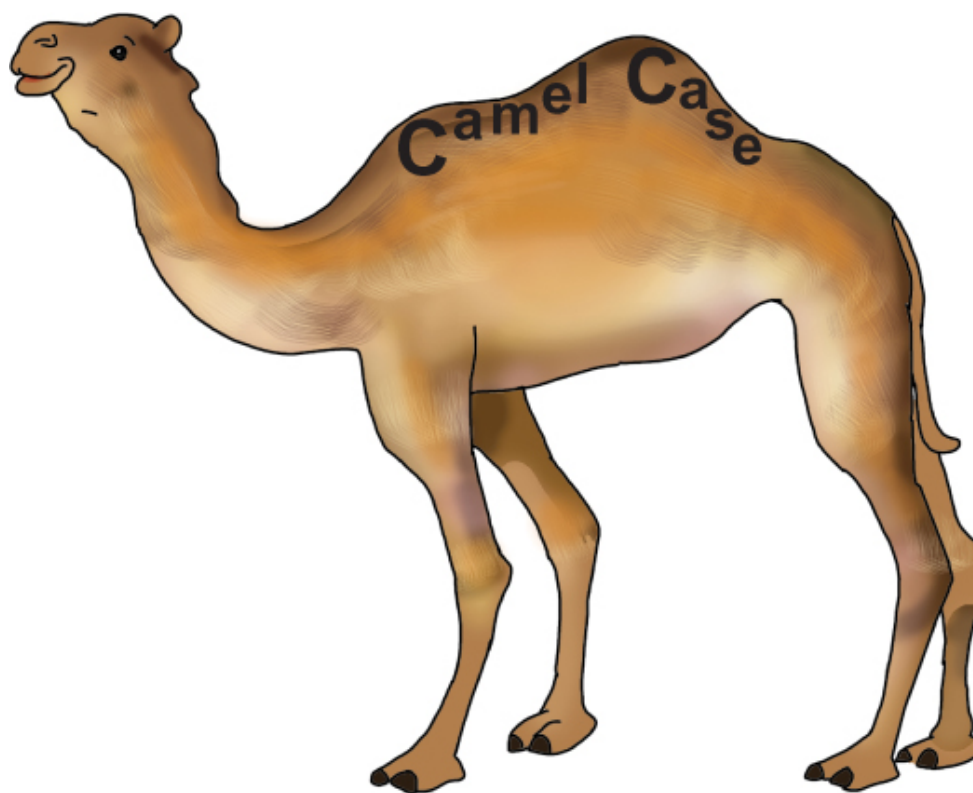


Figure 2.3: The origin of CamelCase.

Having defined the variable names in [Listing 2.1](#), we can use them to concatenate the first and last names, while also inserting a space in between ([Listing 2.2](#)).

Listing 2.2: Concatenating string variables (and a string literal).

```
> firstName + " " + lastName;  
'Michael Hartl'
```

By the way, the use of **let** in [Listing 2.1](#) is characteristic of modern JavaScript (often referred to as ES6 because of the significant upgrade represented by version 6 of the ECMAScript standard ([Box 1.2](#))). In this book, we'll always use **let** (or the closely related **const**, which we'll first see in [Section 4.2](#)) for variable assignment, but you should be aware that the use of the nearly equivalent **var** is still *extremely* common ([Listing 2.3](#)), so it's important to understand both.

Listing 2.3: Using the slightly outdated **var** to assign variables.

```
var firstName = "Michael";  
var lastName  = "Hartl";
```

(You shouldn't type in [Listing 2.3](#); it's shown only for purposes of illustration.)

2.2.1 The Backtick Syntax

Another way to build up strings is via *interpolation* using a special ES6 backtick syntax known as *template literals*:

```
> `${firstName} is my first name.`  
'Michael is my first name.'
```

Here we have a string enclosed in backticks ``...``, with the variable to be interpolated introduced with a dollar sign `$` and enclosed in curly braces `{...}`. JavaScript automatically inserts, or *interpolates*, the value of the variable **firstName** into the string at the appropriate place.⁴

4. Programmers familiar with Perl or PHP should compare this to the automatic interpolation of dollar sign variables in expressions like `"Michael $lastName"`.

We can use the backtick syntax to replicate the result of [Listing 2.2](#), as shown in [Listing 2.4](#).

Listing 2.4: Concatenation review, then interpolating with backticks.

```
> firstName + " " + lastName;    // Concatenation, with a space  
in between  
'Michael Hartl'  
> `${firstName} ${lastName}`;    // The equivalent interpolation  
'Michael Hartl'
```

The two expressions shown in [Listing 2.4](#) are equivalent, but I generally prefer the interpolated version because having to add the single space " " in between strings feels a bit awkward.

2.2.2 Exercises

1. What happens if you use **let** a second time with the same variable name? What if you use **var** instead?
2. Assign variables **city** and **state** to your current city and state of residence. (If residing outside the U.S., substitute appropriate analogues.) Using interpolation, print a string consisting of the city and state separated by a comma and a space, as in “Los Angeles, CA”.
3. Repeat the previous exercise but with the city and state separated by a tab character.

2.3 Printing

As we saw in [Section 1.3](#) and subsequent sections, the JavaScript way to print a string to the screen is to use the **console.log** function:

```
> console.log("hello, world!");    // Print output
hello, world!
```

This function operates as a *side effect*, which refers to anything a function does other than returning a value. In particular, the expression

```
console.log("hello, world!");
```

prints the string to the screen and then returns nothing. This is why some consoles display **undefined** after the printed value ([Figure 2.4](#)). We’ll generally omit **undefined** when showing results in the REPL, but it’s good to distinguish between functions that return values (almost all of them) and those like **console.log** that operate using side effects.

```
> console.log("hello, world!");  
hello, world!  
undefined
```

Figure 2.4: An undefined return value in Node.

In contrast to many other languages—whose print functions are things like **print**, **printf** (“print format”), and **puts** (“put string”)—the print function in JavaScript is rather long and cumbersome, requiring calling a method on the **console** object and using the rather unintuitive name **log**. This is due to the origins of JavaScript as a language designed specifically to run inside web browsers, rather than being designed as a general-purpose programming language.

The name **console.log** is a hint of its original purpose: to write a log to the browser console—a task at which it still excels, and which is useful in debugging. For example, we can write to the index page’s console log by adding a line inside the **script** tag, as shown in [Listing 2.5](#).

Listing 2.5: Writing to the console log.
index.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Learn Enough JavaScript</title>  
    <meta charset="utf-8">  
    <script>  
      alert("hello, world!");  
      console.log("This page contains a friendly greeting.");  
    </script>  
  </head>  
  <body>  
    <h1>Hello, world!</h1>  
    <p>This page includes an alert written in JavaScript.</p>  
  </body>  
</html>
```

The result is that the index page (after displaying the alert) logs the message to the console, as shown in [Figure 2.5](#).

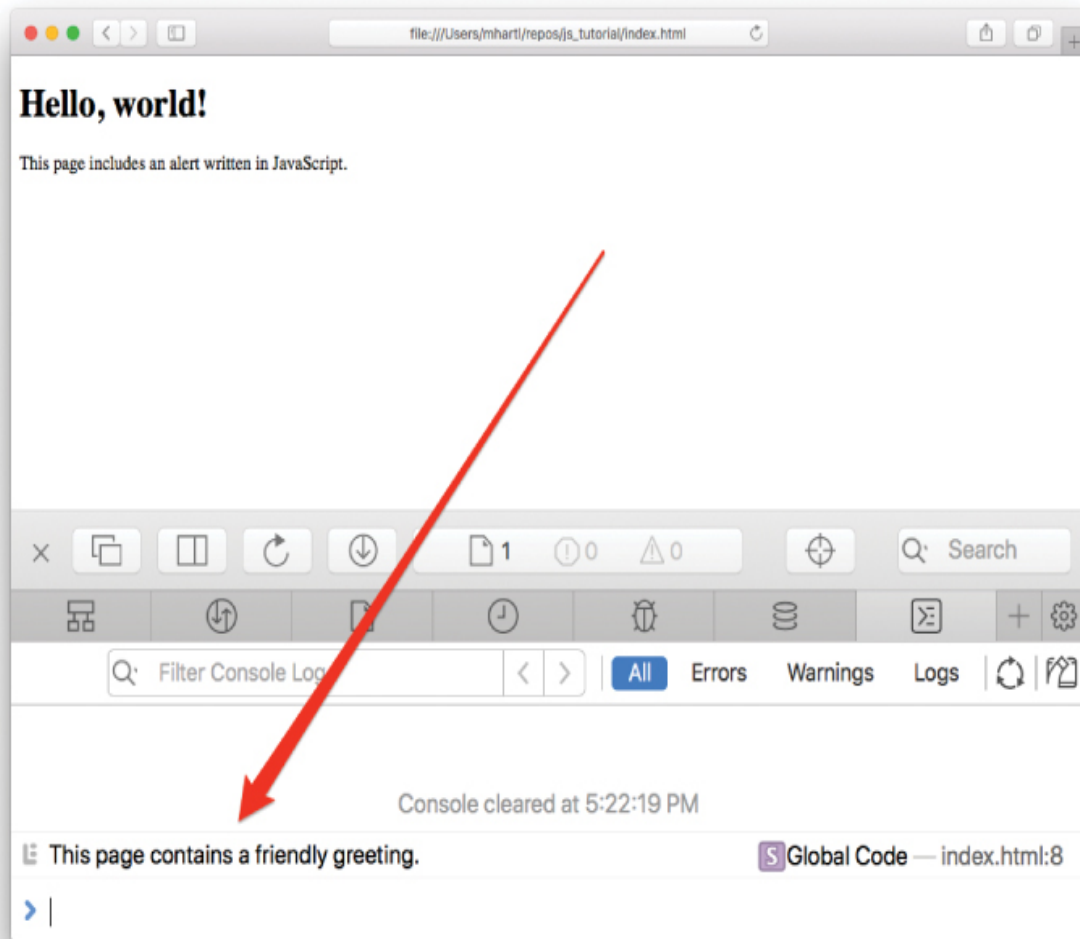


Figure 2.5: The console log message on the index page.

Finally, it's worth noting that (as seen briefly in [Section 1.4.1](#)) the default behavior for **console.log** is to insert a space:

```
> console.log(firstName, lastName);  
Michael Hartl
```

This does you little good if you actually want a string representing the combination (as in [Section 2.2](#)), but it does mean you can omit the concatenation or interpolation if all you're interested in is the output.

2.3.1 Exercises

1. Define **firstName** and **lastName** variables (as in [Section 2.2](#)) inside **index.html**, and output them in the browser console using **console.log**.

2.4 Properties, Booleans, and Control Flow

Almost everything in JavaScript, including strings, is an object. This means that we can get useful information about strings and do useful things with them using the dot notation introduced in [Section 1.3.1](#).

We'll start by accessing a string *property* (also called an *attribute*), which is a piece of data attached to an object. In particular, in the console we can use the **length** property to find the number of characters in a string:

```
$ node
> "badger".length;    // Accessing the "length" property of a string
6
> "".length           // The empty string has zero length.
0
```

As it happens, **length** is the *only* property of string objects, as you can verify using the MDN entry on String (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String) and using your browser's “Find” capability to search for the string “properties”.

The **length** property is especially useful in comparisons, such as checking the length of a string to see how it compares to a particular value (note that the REPL supports “up arrow” to retrieve previous lines, just like the command-line terminal):

```
> "badger".length > 3;
true
> "badger".length > 6;
false
> "badger".length >= 6;
true
> "badger".length < 10;
true
> "badger".length == 6;
true
```

The last line uses the equality comparison operator `==`, which JavaScript shares with many other languages, but there's a huge gotcha:

```
> "1" == 1;    // Uh, oh!  
true
```

In other words, JavaScript considers the string `"1"` to be equal to the number `1`, at least when using `==` to do the comparison.

As programming languages go, this behavior is unusual, so it can be a source of frustrating bugs for people coming to JavaScript from other languages. In order to avoid confusion, it's best to use *triple equals* instead:

```
> "1" === 1;   // This is probably what you want.  
false
```

Throughout the rest of this tutorial, we'll always do equality comparisons with `===`.

The return values in the comparisons above, which are always either `true` or `false`, are known as *boolean* values, after mathematician and logician George Boole ([Figure 2.6](#)).⁵

⁵ Image courtesy of Yogi Black/Alamy Stock Photo.

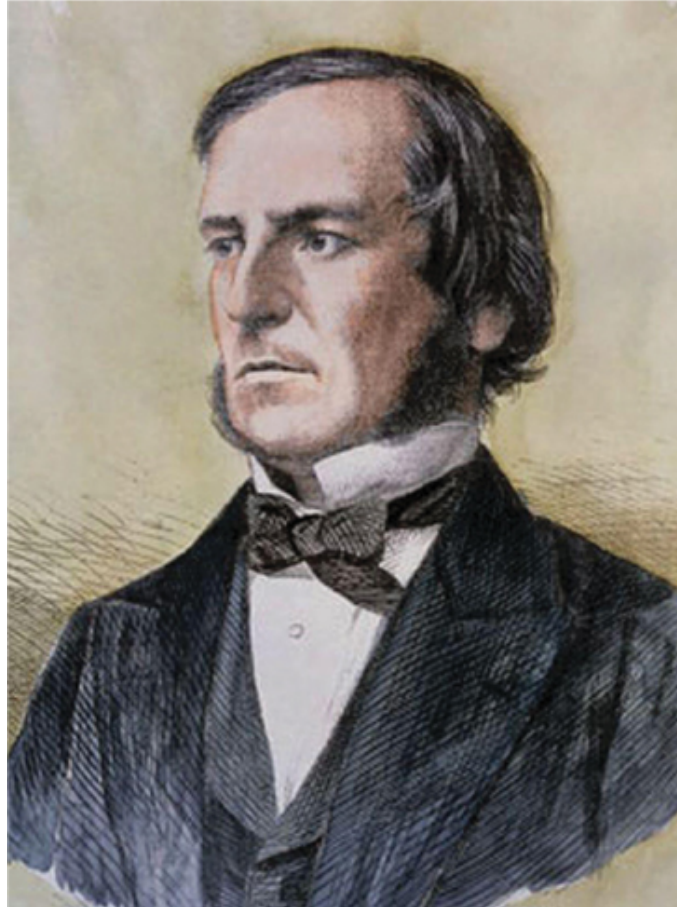


Figure 2.6: True or false? This is a picture of George Boole.

Boolean values are especially useful for *control flow*, which lets us take actions based on the result of a comparison ([Listing 2.6](#)).

Listing 2.6: Control flow with **if**.

```
> let password = "foo";  
> if (password.length < 6) {  
  "Password is too short."  
}  
'Password is too short.'
```

Note in [Listing 2.6](#) that the comparison after **if** is in parentheses, and the string is inside curly braces `{...}`.⁶ We also followed a consistent indentation convention, which is irrelevant to JavaScript but is important for human readers of the code ([Box 2.3](#)).

6. Such curly braces are characteristic of C-like languages, that is, languages with syntax similar to that of the C programming language.

Box 2.3: Code Formatting

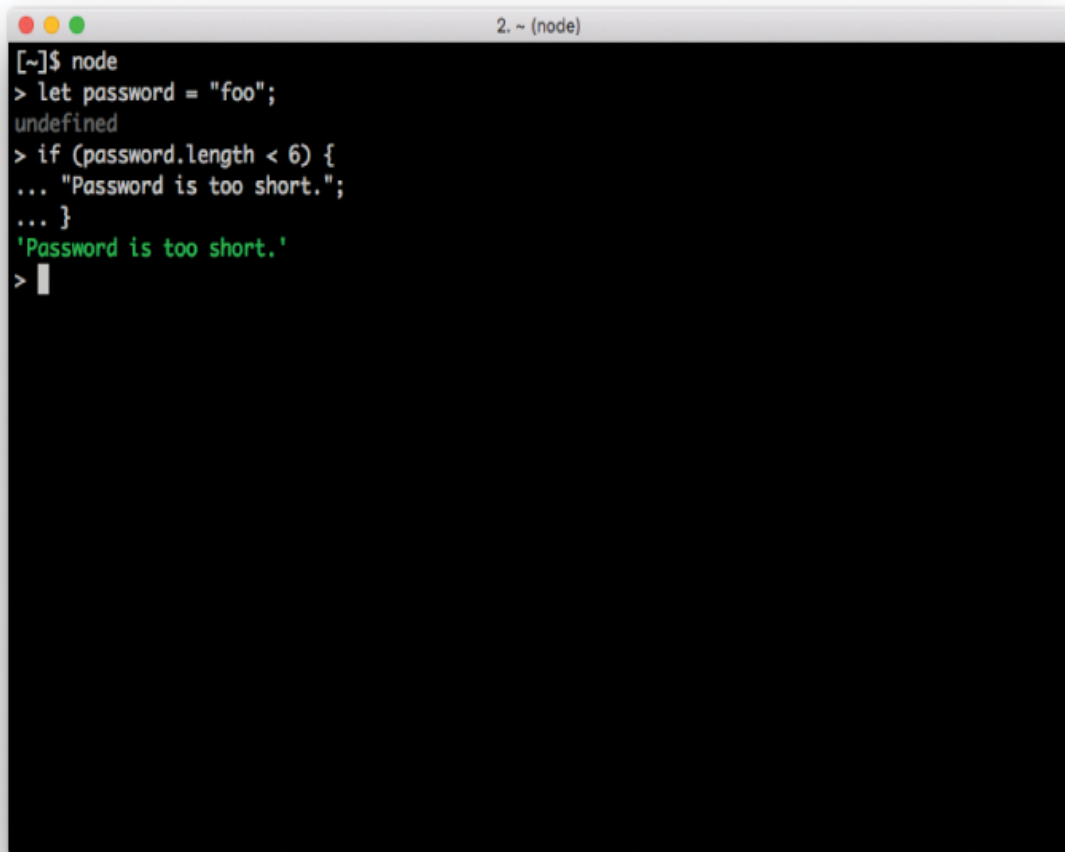
The code samples in this tutorial, including those in the REPL, are designed to show how to format JavaScript in a way that maximizes readability and code comprehension. The programs executing JavaScript, whether Node or the browser itself, don't care about these aspects of the code, but human developers do.

While exact styles differ, here are some general guidelines for good code formatting:

- *Indent code to indicate block structure.* Pretty much every time you see an opening curly brace {, you'll end up indenting the subsequent line. (Some text editors even do this automatically.)
- *Use two spaces (typically via emulated tabs (https://www.learnenough.com/text-editor-tutorial/advanced_text_editing#sec-indenting_and_dedenting)) for indentation.* Many developers use four or even eight spaces, but I find that two spaces are enough to indicate block structure visually while conserving scarce horizontal space.
- *Add newlines to indicate logical structure.* One thing I particularly like to do is add an extra newline after a series of **let** and **const** declarations, in order to give a visual indication that the setup is done and the real coding can begin. An example appears in [Listing 4.6](#).
- *Limit lines to 80 characters (also called “columns”).* This is an old constraint, one that dates back to the early days of 80-character-width terminals. Many modern developers routinely violate this constraint, considering it outdated, but in my experience the 80-character limit is a good source of discipline, and will save your neck when using command-line programs like `less` (or when using your code in a document with more stringent width requirements, such as a book (<https://www.learnenough.com/courses#>)). A line that breaks 80 characters is a hint that you should introduce a new variable name, break an operation into multiple steps, etc., to make the code clearer for anyone reading it.

We'll see several examples of more advanced code-formatting conventions as we proceed throughout the rest of this tutorial.

To reinforce good code-formatting practices, I'll generally format code in the Node REPL the same way I would in a file, but it's important to note that this is not necessarily what you will see in the actual REPL. For example, many implementations of the Node REPL automatically insert triple dots `...` after an opening curly brace (Figure 2.7) to indicate a new block of code. This difference is not cause for concern, and you should use your technical sophistication (Box 1.1) to resolve the discrepancy between the REPL code samples and the exact appearance on your system.

A screenshot of a Node.js REPL window titled "2. ~ (node)". The window has a dark background with light green text. The prompt is "[~]\$ node". The user enters "> let password = 'foo';". The output is "undefined". The user enters "> if (password.length < 6) {" followed by "... 'Password is too short.';" on the next line, and "... }" on the third line. The output is "'Password is too short.'" on the first line, and then the prompt "> " on the second line. The triple dots are used to indicate a new block of code.

```
[~]$ node
> let password = "foo";
undefined
> if (password.length < 6) {
... "Password is too short.";
... }
'Password is too short.'
>
```

Figure 2.7: The appearance of code in the REPL doesn't necessarily follow indentation conventions.

We can add a second behavior using **else**, which serves as the default result if the first comparison is **false** (Listing 2.7).

Listing 2.7: Control flow with **if** and **else**.

```
> password = "foobar";
> if (password.length < 6) {
  "Password is too short.";
} else {
  "Password is long enough.";
}
'Password is long enough.'
```

The first line in Listing 2.7 redefines **password** by assigning it a new value (with no **let** required since it was already defined before). After reassignment, the **password** variable has length 6, so **password.length < 6** is **false**. As a result, the **if** part of the statement (known as the *if branch*) doesn't get evaluated; instead, JavaScript evaluates the **else** branch, resulting in a message indicating that the password is long enough.

2.4.1 Combining and Inverting Booleans

Booleans can be combined or inverted using the **&&** (“and”), **||** (“or”), and **!** (“bang” or “not”) operators.

Let's start with **&&**. When comparing two booleans with **&&**, *both* have to be **true** for the combination to be **true**. For example, if I said I wanted both french fries *and* a baked potato, the only way the combination could be true is if I could answer “yes” (true) to both of the questions “Do you want french fries?” and “Do you want a baked potato?” If my answer to either of those is false, then the combination must be false as well. The resulting combinations of possibilities, called a *truth table*, appears in Listing 2.8.

Listing 2.8: The truth table for **&&** (“and”).

```
> true && true
true
> false && true
false
> true && false
false
> false && false
false
```

We can apply this to a conditional as shown in [Listing 2.9](#).

Listing 2.9: Using the `&&` operator in a conditional.

```
> let x = "foo";
> let y = "";
> if (x.length === 0 && y.length === 0) {
  "Both strings are empty!";
} else {
  "At least one of the strings is nonempty.";
}
'At least one of the strings is nonempty.'
```

In [Listing 2.9](#), `y.length` is in fact `0`, but `x.length` isn't, so the combination is **false** (in agreement with [Listing 2.8](#)), so JavaScript evaluates the **else** branch.

In contrast to `&&`, `||` lets us take action if *either* comparison (or both) is true ([Listing 2.10](#)).

Listing 2.10: The truth table for `||` (“or”).

```
> true || true
true
> true || false
true
> false || true
true
> false || false
false
```

We can use `||` in a conditional as shown in [Listing 2.11](#).

Listing 2.11: Using the `||` operator in a conditional.

```
> if (x.length === 0 || y.length === 0) {
  "At least one of the strings is empty!";
} else {
  "Neither of the strings is empty.";
}
'At least one of the strings is empty!'
```

Note from [Listing 2.10](#) that `||` isn't *exclusive*, meaning that the result is true even when *both* statements are true. This stands in contrast to colloquial usage, where a

statement like “I want fries or a baked potato” implies that you want either fries or a baked potato, but you don’t want both (Figure 2.8).⁷

7. Image courtesy of Rikaphoto/Shutterstock.



Figure 2.8: Turns out I only wanted fries.

In addition to `&&` and `||`, JavaScript supports *negation* via the “not” operator `!` (often pronounced “bang”), which just converts **true** to **false** and **false** to **true** (Listing 2.12).

Listing 2.12: The truth table for `!`.

```
> !true
false
> !false
true
```

We can use `!` in a conditional as shown in Listing 2.13.

Listing 2.13: Using the `!` operator in a conditional.

```
> if (!(x.length === 0)) {  
  "x is not empty."  
} else {  
  "x is empty."  
}  
'x is not empty.'
```

The code in [Listing 2.13](#) is valid JavaScript, as it simply negates the test `x.length === 0`, yielding `true`:

```
> (!(x.length === 0))  
true
```

In this case, though, it's more common to use `!==` (“not equals”):

```
> if (x.length !== 0) {  
  "x is not empty."  
} else {  
  "x is empty."  
}  
'x is not empty'
```

2.4.2 Bang Bang

Not all booleans are the result of comparisons, and in fact every JavaScript object has a value of either `true` or `false` in a boolean context. We can force JavaScript to use such a boolean context with `!!` (pronounced “bang bang”); because `!` converts between `true` and `false`, using *two* exclamation points returns us back to the original boolean:

```
> !!true  
true  
> !!false  
false
```

Using this trick allows us to see that a string like `"foo"` is `true` in a boolean context:

```
> !!"foo"  
true
```

As it happens, the empty string is *false* in a boolean context:⁸

8. This is the sort of detail that varies from language to language. In Ruby, for example, even the empty string is **true** in a boolean context.

```
> !!""  
false
```

As a result, we can rewrite code like [Listing 2.9](#) more compactly by omitting the length comparison (while negating **x** and **y**), as shown in [Listing 2.14](#).

Listing 2.14: Using a conditional to force a boolean context.

```
> if (!x && !y) {  
    "Both strings are empty!";  
} else {  
    "At least one of the strings is nonempty.";  
}  
'At least one of the strings is nonempty.'
```

2.4.3 Exercises

1. If **x** is **"foo"** and **y** is **""** (the empty string), what is the value of **x && y**? Verify using the “bang bang” notation that **x && y** is false in a boolean context. *Hint:* When applying **!!** to a compound expression, wrap the whole thing in parentheses.
2. What is **x || y**? What is it in a boolean context? Rewrite [Listing 2.14](#) to use **x || y**, ensuring that the result is the same. (*Hint:* Switch the order of the strings.)

2.5 Methods

As noted in [Section 2.4](#), JavaScript string objects have only one property (**length**), but they support a wide variety of methods.⁹ In the language of object-oriented programming, a particular string, or *string instance*, is said to “respond to” a particular method, indicated using the dot notation first seen in [Section 1.3.1](#).

⁹ Recall from [Section 1.3.1](#) that a method is a particular kind of function, one attached to an object and invoked using the dot notation.

For example, strings respond to the instance method **toLowerCase()**, which (surprise!) converts the string to all lowercase letters ([Figure 2.9](#)):¹⁰

10. Image courtesy of Pavel Kovaricek/Shutterstock.



Figure 2.9: This honey badger used to be a HONEY BADGER, but he don't care.

```
$ node  
> "HONEY BADGER".toLowerCase();  
'honey badger'
```

This is the sort of method that could be useful, for example, when standardizing on lowercase letters in an email address:¹¹

11. If you've exited and re-entered your Node console, `firstName` might no longer be defined, as such definitions don't persist from session to session. If this is the case, apply your technical sophistication ([Box 1.1](#)) to figure out what to do.

```
> let username = firstName.toLowerCase();  
> `${username}@example.com`; // Sample email address  
'michael@example.com'
```

Note that, in contrast to the `length` property, a method has to be called with arguments, even if there aren't any. That's why

```
toLowerCase()
```

ends with opening and closing parentheses: **toLowerCase** is a function that takes zero arguments. Also note that official JavaScript string methods follow the same capitalization convention (CamelCase with a lowercase letter to start) that we introduced ourselves in [Section 2.2](#).

As you might be able to guess, JavaScript supports the opposite operation as well; before looking at the example below, see if you can guess the method for converting a string to uppercase ([Figure 2.10](#)).¹²

¹². Image courtesy of arco1/123RF.



Figure 2.10: Early typesetters kept large letters in the “upper case” and small letters in the “lower case”.

I’m betting you got the right answer:

```
> lastName.toUpperCase();  
'HARTL'
```

Being able to guess answers like this is a hallmark of technical sophistication, but as noted in [Box 1.1](#) another key skill is being able to use the documentation. In particular, the Mozilla Developer Network page on **String** objects has a long list of useful string instance methods.¹³ Let’s take a look at some of them ([Figure 2.11](#)).

¹³. You can find such pages by going directly to the MDN website (<https://developer.mozilla.org/en-US/docs/Web/JavaScript>), but the truth is that I nearly always find such pages by Googling things like “javascript string”.

Methods

Methods unrelated to HTML

`String.prototype.charAt()`

Returns the character (exactly one UTF-16 code unit) at the specified index.

`String.prototype.charCodeAt()`

Returns a number that is the UTF-16 code unit value at the given index.

`String.prototype.codePointAt()`

Returns a nonnegative integer Number that is the code point value of the UTF-16 encoded code point starting at the specified index.

`String.prototype.concat()`

Combines the text of two strings and returns a new string.

`String.prototype.includes()`

Determines whether one string may be found within another string.

Figure 2.11: Some JavaScript string methods.

Inspecting the methods in [Figure 2.11](#), we see on the bottom some code that looks like this:

```
String.prototype.includes()
```

followed by a brief description. What does **`String.prototype`** mean here? We'll find out in Chapter 7, but the real answer is *we don't have to know exactly what it means to use the documentation*. Selective ignorance is classic technical sophistication.

Clicking through on the **`String.prototype.includes()`** link and scrolling down shows us a bunch of examples ([Figure 2.12](#)). Notice that, as prophesied in [Section 2.2](#), the use of **`var`** instead of **`let`** is very common; being able to tolerate these slight mismatches is yet another application of technical sophistication.

Examples

Using `includes()`

```
1 var str = 'To be, or not to be, that is the question.';
2
3 console.log(str.includes('To be'));      // true
4 console.log(str.includes('question'));   // true
5 console.log(str.includes('nonexistent')); // false
6 console.log(str.includes('To be', 1));    // false
7 console.log(str.includes('TO BE'));       // false
```

Figure 2.12: Examples for the string `includes()` method.

Let's try out the examples shown in [Figure 2.12](#), with the following modifications:

1. Use `let` instead of `var`.
2. Use `soliloquy` instead of `str`.
3. Use double-quoted strings instead of single-quoted strings.
4. Change the quote to use a colon as in the original ([Figure 2.13](#)).¹⁴

¹⁴. Image courtesy of Everett Collection/Shutterstock.



Figure 2.13: Hamlet, Prince of Denmark, asks: “To be, or not to be, that is the question.”

5. Omit the use of `console.log`.
6. Omit the current comments, while adding some of our own.

The result in a Node REPL looks something like [Listing 2.15](#).

Listing 2.15: Includes or does not include? That is the question.

```
> let soliloquy = "To be, or not to be, that is the question:";
> soliloquy.includes("To be");           // Does it include the
substring "To be"?
true
> soliloquy.includes("question");       // What about "question"?
true
> soliloquy.includes("nonexistent");    // This string doesn't
appear.
false
> soliloquy.includes("TO BE");          // String inclusion is case-
sensitive.
false
> soliloquy.includes("To be", 1);       // Can you guess what this
```

```
one means?  
false  
> soliloquy.includes("o be", 1);    // A hint for the previous  
one  
true
```

Of the lines in [Listing 2.15](#), the only two that you might not be able to figure out right away are the last two. You'll find the solution to this mystery, as well as pointers to some other common string methods, in the section exercises.

2.5.1 Exercises

1. Write the JavaScript code to test whether the string “hoNeY BaDGeR” includes the string “badger” without regard to case.
2. What does `includes(string, i)` do for any integer `i`? *Hint*: Counting in JavaScript starts at `0` rather than `1`.

2.6 String Iteration

Our final topic on strings is *iteration*, which is the practice of repeatedly stepping through an object one element at a time. Iteration is a common theme in computer programming, and we'll see some other examples later in this tutorial ([Section 3.5](#) and [Section 5.4](#)). We'll also see how one sign of your growing power as a developer is learning how to *avoid* iteration entirely (as discussed in Chapter 6 and Section 8.5).

In the case of strings, we'll be learning how to iterate one *character* at a time. There are two main prerequisites to this: First, we need to learn how to access a particular character in a string, and second, we need to learn how to make a *loop*.

We can figure out how to access a particular string character by consulting the list of **String** methods (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String), which includes the following entry:

```
String.prototype.charAt()  
Returns the character (exactly one UTF-16 code unit) at the specified  
index.
```

Drilling down into the documentation (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/charAt) for the method itself, we see from the examples what **charAt** and “index” mean in this context. Using the **soliloquy** string from [Section 2.5](#), we can illustrate this as shown in [Listing 2.16](#).

Listing 2.16: Investigating the behavior of **charAt**.

```
> console.log(soliloquy);    // Just a reminder of what the string
is
To be, or not to be, that is the question:
> soliloquy.charAt(0);
'T'
> soliloquy.charAt(1);
'o'
> soliloquy.charAt(2);
' '

```

We see in [Listing 2.16](#) that **charAt(0)** returns the first character, **charAt(1)** returns the second, and so on. (We’ll discuss this possibly counter-intuitive numbering convention, called “zero-offset”, further in [Section 3.1](#).) Each number **0**, **1**, **2**, etc., is called an *index* (plural *indexes* or *indices*).

Now let’s look at our first example of a loop. In particular, we’ll use a **for** loop that defines an index value **i** and increments its value until it reaches some maximum ([Listing 2.17](#)).

Listing 2.17: A simple **for** loop.

```
> for (let i = 0; i < 5; i++) {
  console.log(i);
}
0
1
2
3
4

```

This sort of loop (with only minor variations in syntax) is common across an astonishing variety of programming languages, from C and C++ to Java, Perl, PHP, and (as we’ve just seen) JavaScript. [Listing 2.17](#) shows how, after using **let** to create **i** and set it to **0**, the index variable gets incremented by **1** until it reaches **5**, at which point **i < 5** is **false** and the loop stops. The notation **i++**,

meanwhile, is an *increment* statement that bumps up the value of **i** by one at a time.

If you find the code in [Listing 2.17](#) confusing or ugly, you’re in good company. I consider it a hallmark of good programming to *avoid* using **for** loops as much as possible, preferring instead **forEach** loops ([Section 5.4](#)) or avoiding loops entirely using functional programming (Chapter 6 and [Section 8.5](#)). As computer scientist (and personal friend) Mike Vanier ([Figure 2.14](#)) once put it in an email to Paul Graham:



Figure 2.14: Just a few more **for** loops and Mike Vanier will be a millionaire.

This [tedious repetition] grinds you down after a while; if I had a nickel for every time I’ve written “for (i = 0; i < N; i++)” in C I’d be a millionaire.

Note how the **for** loop syntax in Mike’s email is almost identical to that in [Listing 2.17](#); the only differences are the absence of **let** and the use of **N**, which we can infer from context represents some upper bound on the loop’s index.

We’ll see how to avoid getting ground down starting in Chapter 6, but for now [Listing 2.17](#) is the best we can do.

Let’s combine [Listing 2.16](#) and [Listing 2.17](#) to iterate through all the characters in the first line of Hamlet’s famous soliloquy. The only new thing we need is the index for when the loop should stop. In [Listing 2.17](#), we hard-coded the upper limit (**i < 5**), and we could do the same here if we wanted. The **soliloquy** variable is a bit long to count the characters by hand, though, so let’s ask JavaScript to tell us using the **length** property ([Section 2.4](#)):

```
> soliloquy.length  
42
```

This exceptionally auspicious result suggests writing code like this:

```
for (let i = 0; i < 42; i++) {  
  console.log(soliloquy.charAt(i));  
}
```

This code will work, and it is in perfect analogy with [Listing 2.17](#), but it also raises a question: Why hard-code the length when we can just use the **length** property in the loop itself?

The answer is that we shouldn't, and when looping it's common practice to use the **length** property whenever possible. The resulting improved **for** loop (with result) appears in [Listing 2.18](#).

Listing 2.18: Combining **charAt** and a **for** loop.

```
> for (let i = 0; i < soliloquy.length; i++) {  
  console.log(soliloquy.charAt(i));  
}  
T  
o  
b  
e  
.  
.  
.  
t  
i  
o  
n  
:  
:
```

As noted above, **for** loops are best avoided if at all possible, but this less elegant style of looping is still an excellent place to start. As we'll see in Chapter 8, one powerful technique is to write a *test* for the functionality we want, then get it passing any way we can, and then *refactor* the code to use a more elegant method. The second step in this process (called *test-driven development*, or TDD) often involves writing inelegant but easy-to-understand code—a task at which the humble **for** loop excels.

2.6.1 Exercises

1. Use **let** to define a variable **N** that's equal to the length of **soliloquy**, and show that the code in Mike Vanier's **for** loop actually works in JavaScript

exactly as written. (In particular, you can sometimes get away with omitting **let**, though this isn't a good practice.)

2. Show that you can replace the **charAt** method in [Listing 2.18](#) with a literal bracket notation, like this: **soliloquy[i]**.

Chapter 3. Arrays

In [Chapter 2](#), we saw that strings can be thought of as sequences of characters in a particular order. In this chapter, we'll learn about the *array* data type, which is the general JavaScript container for arbitrary elements in a particular order. We'll start by explicitly connecting strings and arrays via the String **split** method ([Section 3.1](#)), and then learn about various array methods throughout the rest of the chapter.

3.1 Splitting

So far we've spent a lot of time understanding strings, and there's a natural way to get from strings to arrays via the **split** method:

```
> "ant bat cat".split(" ");    // Split a string into a three-  
element array.  
[ 'ant', 'bat', 'cat' ]
```

We see from this result that **split** returns a list of the strings that are separated from each other by a space in the original string.

Splitting on space is one of the most common operations, but we can split on nearly anything else as well:

```
> "ant,bat,cat".split(",");  
[ 'ant', 'bat', 'cat' ]  
> "ant, bat, cat".split(", ");  
[ 'ant', 'bat', 'cat' ]  
> "antheybatheycat".split("hey");  
[ 'ant', 'bat', 'cat' ]
```

We can even split a string into its component characters by splitting on the empty string:

```
> "badger".split("")  
[ 'b', 'a', 'd', 'g', 'e', 'r' ]
```

We'll put this basic technique to good use in [Section 5.3](#) (and we'll also discover it has an important limitation).

Finally, it's worth noting that splitting supports *regular expressions*, which are covered in [Section 4.3](#).

3.1.1 Exercises

1. Assign **a** to the result of splitting the string “A man, a plan, a canal, Panama” on comma-space. How many elements does the resulting array have?
2. Can you guess the method to reverse **a** in place? (Google around if necessary.)

3.2 Array Access

Having connected strings with arrays via the **split** method, we'll now discover a second close connection as well. Let's start by assigning a variable to an array of characters created using **split**:

```
> let a = "badger".split("");
```

We can access particular elements of **a** using a bracket notation that's common to a huge number of different languages, as seen in [Listing 3.1](#).

Listing 3.1: Array access with the bracket notation.

```
> a[0];  
'b'  
> a[1];
```

```
'a'  
> a[2];  
'd'
```

Does [Listing 3.1](#) look a little familiar? It’s the same basic relationship between characters and numerical index that we saw with the **String#charAt** method in [Listing 2.16](#). (The notation in the previous sentence indicates that **charAt** is an *instance method*, i.e., a method on string instances.) In fact, the bracket notation actually works directly on strings:

```
> "badger"[0];  
'b'  
> "badger"[1];  
'a'
```

We see from [Listing 3.1](#) that, as with strings, arrays are *zero-offset*, meaning that the “first” element has index **0**, the second has index **1**, and so on. This convention can be confusing, and in fact it’s common to refer to the initial element for zero-offset arrays as the “zeroth” element as a reminder that the indexing starts at **0**. This convention can also be confusing when using multiple languages (some of which start array indexing at **1**), as illustrated in the xkcd comic strip “Donald Knuth” (<https://m.xkcd.com/163/>).¹

1. This particular xkcd strip takes its name from renowned computer scientist Donald Knuth (pronounced “kuh-NOOTH”), author of *The Art of Computer Programming* and creator of the TEX typesetting system used to prepare many technical documents, including this one.

So far we’ve dealt exclusively with arrays of characters, but JavaScript arrays can contain all kinds of elements:

```
> a = ["badger", 42, soliloquy.includes("To be")];  
[ 'badger', 42, true ]  
> a[2];  
true  
> a[3];  
undefined
```

We see here that the square bracket access notation works as usual for an array of mixed types, which shouldn’t come as a surprise. We also see that

trying to access an array index outside of the defined range returns **undefined** (a value which we saw before in the context of **console.log** (Figure 2.4)). This might be a surprise if you have previous programming experience, since many languages raise an error if you try to access an element that's out of range, but JavaScript is more tolerant in this regard.

3.2.1 Exercises

1. Write a **for** loop to print out the characters obtained from splitting “honey badger” on the empty string.
2. See if you can guess the value of **undefined** in a boolean context. Use **!!** to confirm.

3.3 Array Slicing

In addition to supporting the bracket notation described in [Section 3.2](#), JavaScript supports a technique known as *array slicing* for accessing multiple elements at a time. In anticipation of learning to *sort* in [Section 3.4](#), let's redefine our array **a** to have purely numerical elements:

```
> a = [42, 8, 17, 99];  
[ 42, 8, 17, 99 ]
```

The simplest way to slice an array is to provide only one argument, which returns all the elements in the array from that index on. For example, for an array with four elements, **slice(1)** returns the second, third, and fourth ones (recall that the “first” or zeroth element has index **0**):

```
> a.slice(1);  
[ 8, 17, 99 ]
```

We can also slice from one index to another:

```
> a.slice(1, 3);  
[ 8, 17 ]
```

Slicing gives us an easy way to perform a common task, which is to access the last element in an array. Arrays, like strings, have a **length** property, so we could find the last element like this:

```
> a.length;  
4  
> a[a.length-1];  
99
```

This can get a little messy if the variable name is long, though, which often happens in bigger projects that have lots of variables:

```
> let aMuchLongerArrayName = a;  
> aMuchLongerArrayName[aMuchLongerArrayName.length - 1];  
99
```

This leads us to a second method for picking off the last element, which is to use **slice** with a *negative* number, which counts from the end:

```
> aMuchLongerArrayName.slice(-1);  
[ 99 ]
```

This is an array with one element, so we can select the element itself using the bracket notation:

```
> aMuchLongerArrayName.slice(-1)[0];  
99
```

A final common case is where we want to access the final element and remove it at the same time. We'll cover the method for doing this in [Section 3.4.2](#).

3.3.1 Exercises

1. Define an array with the numbers 1 through 10. Use slicing and **length** to select the third element through the third-to-last. Accomplish the same task using a negative index.
2. Show that strings also support the **slice** method by selecting just **bat** from the string **"ant bat cat"**. (You might have to experiment a little to get the indices just right.)

3.4 More Array Methods

In addition to the **slice** method seen in [Section 3.3](#), arrays respond to a wealth of other methods. As usual, the documentation (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array) is a good place to go for details.

As with strings, arrays respond to an **includes** method to test for element inclusion:

```
> a;  
[ 42, 8, 17, 99 ]  
> a.includes(42);           // Test for element inclusion.  
true  
> a.includes("foo");  
false
```

3.4.1 Sorting and Reversing

You can also sort an array in place—an excellent trick that in ye olden of C often required a custom implementation. In JavaScript, we just call **sort()**:

```
> a.sort();  
[ 17, 42, 8, 99 ]  
> a;           // `a` has changed as the result of  
`sort()`.  
[ 17, 42, 8, 99 ]
```

You might notice something strange here, which is that JavaScript has sorted the elements of the array not according to their numerical values, but rather “alphabetically”, so that **17** comes before **8** because **1** comes before **8** in the ordering scheme (ASCII) used by computers. (We’ll learn how to sort arrays numerically in [Chapter 5](#).)

Another useful method—one we’ll put to good use in developing our palindrome theme starting in [Section 5.3](#)—is the **reverse** method:

```
> a.reverse();
[ 99, 8, 42, 17 ]
> a;                                     // Like `sort()`, `reverse()` mutates the
[ 99, 8, 42, 17 ]                       array.
```

As noted in the comment, methods like **a.sort()** and **a.reverse()** *mutate* the array, meaning they modify it as a side effect of performing their respective actions. This is the sort of behavior that varies from one programming language to the next, so be careful when using similar methods in other languages.

3.4.2 Pushing and Popping

One useful pair of array methods is **push** and **pop**; **push** lets us append an element to the end of an array, while **pop** removes it:

```
> a.push(6);                             // Pushing onto an array (returns new
length)
5
> a;
[ 99, 8, 42, 17, 6 ]
> a.push("foo");
6
> a;
[ 99, 8, 42, 17, 6, 'foo' ]
> a.pop();                               // `pop` returns the value itself
'foo'
> a.pop();
6
> a;
[ 99, 8, 42, 17 ]
```

As noted in the comments, **pop** returns the value of the final element (while removing it as a side effect), but **push** (somewhat counter-intuitively) returns the *length* of the new array. As of this writing, I don't know why (and neither does Stack Overflow).

We are now in a position to appreciate the comment made in [Section 3.3](#) about obtaining the last element of the array, as long as we don't mind mutating it:

```
> let lastElement = a.pop();
> lastElement;
17
> a;
[ 99, 8, 42 ]
> let theAnswerToLifeTheUniverseAndEverything = a.pop();
```

3.4.3 Undoing a Split

A final example of an array method, one that brings us full circle from [Section 3.1](#), is **join**. Just as **split** splits a string into array elements, **join** joins array elements into a string ([Listing 3.2](#)).

Listing 3.2: Different ways to join.

```
> a = ["ant", "bat", "cat", 42];
[ 'ant', 'bat', 'cat', 42 ]
> a.join();                                // Join on default (comma).
'ant,bat,cat,42'
> a.join(", ");                            // Join on comma-space.
'ant, bat, cat, 42'
> a.join(" -- ");                          // Join on double dashes.
'ant -- bat -- cat -- 42'
> a.join("");                              // Join on empty space.
'antbatcat42'
```

Note that **42**, which is an integer, is automatically converted to a string in the join.

3.4.4 Exercises

1. The **split** and **join** methods are almost inverse operations, but not quite. In particular, confirm using **==** (not **===**) that **a.join(" ").split(" ")** in [Listing 3.2](#) is *not* the same as **a**. Why not?
2. Using the array documentation, figure out how to push onto or pop off the *front* of an array. *Hint*: The names aren't intuitive at all, so you might have to work a bit.

3.5 Array Iteration

One of the most common tasks with arrays is iterating through their elements and performing an operation with each one. This might sound familiar, since we solved the exact same problem with strings in [Section 2.6](#), and indeed the solution is virtually the same. All we need to do is adapt the **for** loop from [Listing 2.18](#) to arrays.

We could get there in one step fairly easily, but the connection is even clearer if we first rewrite the string **for** loop using the bracket access notation, which (as we saw in [Section 3.2](#)) works on strings as well. The result for the **soliloquy** string defined in [Listing 2.15](#) is shown in [Listing 3.3](#).

Listing 3.3: Combining string access and a **for** loop.

```
> for (let i = 0; i < soliloquy.length; i++) {  
  console.log(soliloquy[i]);  
}  
T  
o  
  
b  
e  
.  
.  
.  
t  
i  
o  
n  
:
```

The result in [Listing 3.3](#) is exactly the same as that shown in [Listing 2.18](#).

The application of this pattern to arrays should now be clear. All we need to do is replace **soliloquy** with **a**, as shown in [Listing 3.4](#); the rest of the code is identical.

Listing 3.4: Combining array access and a **for** loop.

```
> for (let i = 0; i < a.length; i++) {  
    console.log(a[i]);  
}  
ant  
bat  
cat  
42
```

One thing worth noting here is that the iteration index variable **i** appears in both **for** loops. As you may recall if you completed the exercises in [Section 2.2.2](#), redefining a variable that's already been declared with **let** generally results in an error. Why were we able to reuse **i** in this context?

The answer is that in the context of a **for** loop the *scope* of the variable is restricted to the loop, and disappears when the loop is finished.

That's convenient, but it's not the best way to iterate through arrays, and Mike Vanier still wouldn't be happy ([Figure 3.1](#)). We'll see a cleaner method for iterating through arrays in [Section 5.4](#), and a way of avoiding iteration entirely in Chapter 6.



Figure 3.1: Mike Vanier is still annoyed by typing out **for** loops.

3.5.1 Exercises

1. Show that the identifier **i** is undefined both before and after a **for** loop executes. (You might have to exit and re-enter the Node console.)
2. Define an accumulator variable **total** and combine it with a loop to add all the elements of [Listing 3.4](#). You can use the code in [Listing 3.5](#) to get started (just replace the comment with the proper code). How does the value of **total** compare to **a.join("")**?

Listing 3.5: Skeleton for a total.

```
> let total = "";
> for (let i = 0; i < a.length; i++) {
    // set total equal to the running total plus the current
    element
}
```

Chapter 4. Other Native Objects

Now that we've taken a look at strings and arrays, we'll continue with a tour of some other important JavaScript objects: math, dates, regular expressions, and generic objects.

4.1 Math and Number

Like most programming languages, JavaScript supports a large number of mathematical operations:

```
> 1 + 1;
2
> 2 - 3;
-1
> 2 * 3;
6
> 10/5;
2
> 2/3;
0.6666666666666666
```

Note that the final example here isn't exact; it's a *floating-point* number (also called a *float*), which can't be represented exactly by the computer. But in fact JavaScript has only one numerical type, and even something like **1** or **2** is treated as floating point under the hood. This is convenient for us as programmers, since it means we never have to make distinctions between different kinds of numbers.¹

¹. In contrast to JavaScript, many languages distinguish between integers and floats, which leads to pitfalls like **1.0/2.0** being the expected **0.5**, but **1/2** being **0**.

Many programmers, including me, find it convenient to fire up a REPL and use it as a simple calculator when the need arises. It's not fancy, but it's quick and relatively powerful, and the ability to define variables often comes in handy as well.

4.1.1 More Advanced Operations

JavaScript supports more advanced mathematical operations via a *global object* called **Math**, which has properties and methods for things like mathematical constants, exponentiation (powers),² roots, and trigonometric functions:

2. Adding support for exponentiation with two asterisks ****** is in the works but isn't universally implemented as of this writing.

```
> Math.PI
3.141592653589793
> Math.pow(2, 3);
8
> Math.sqrt(3)
1.7320508075688772
> Math.cos(2*Math.PI)
1
```

There is one gotcha for those coming from high-school (and even college) textbooks that use $\ln$ for the natural logarithm. Like mathematicians and most other programming languages, JavaScript uses **log** instead:

```
> Math.E;
2.718281828459045
> Math.log(Math.E);
1
> Math.log(10);
2.302585092994046
```

Mathematicians typically indicate base-ten logarithms using $\log_{10}$, and JavaScript follows suit with **log10**:

```
> Math.log10(10);
1
> Math.log10(1000000);
6
> Math.log10(Math.E);
0.4342944819032518
```

The **Math** documentation (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math) includes a more comprehensive list of further operations.

4.1.2 Math to String

We discussed in [Chapter 3](#) how to get from strings to arrays (and *vice versa*) using `split` and `join`. Similarly, JavaScript allows us to convert between numbers and strings.

Probably the most common way to convert from a number to a string is using the `toString()` method, as we can see with this useful definition (<https://tauday.com/tau-manifesto>) ([Figure 4.1](#)):³

3. The use of τ to represent the circle constant $6.283185\dots$ was proposed in a math essay I published in 2010 called *The Tau Manifesto*, which also established a math holiday called Tau Day (<https://tauday.com/>), celebrated annually on June 28.

```
> let tau = 2 * Math.PI;  
> tau.toString();  
'6.283185307179586'
```

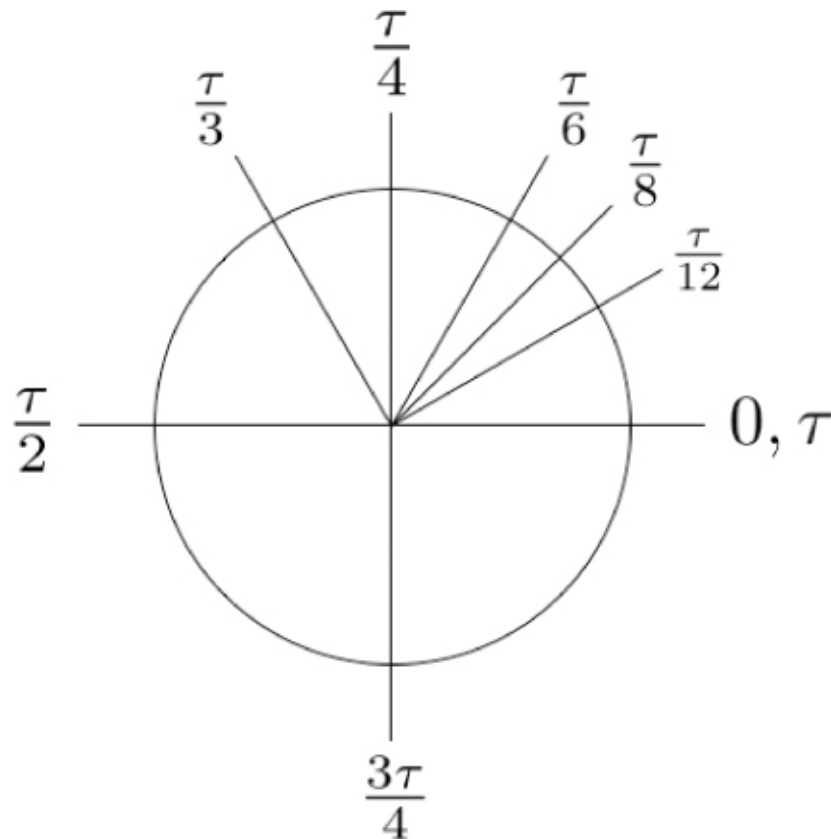


Figure 4.1: Some special angles in terms of $\tau = 2\pi$.

The `toString()` method won't work on a bare integer:

```
> 100.toString();  
100.toString();  
^^^^
```

SyntaxError: Invalid or unexpected token

But it will work if you use an extra dot, so that JavaScript treats the number as a float:

```
> 100.0.toString()  
'100'
```

This is unfortunate behavior, since the string corresponding to **100.0** should more properly be **"100.0"**, but this is a price we pay for JavaScript's lack of a proper integer data type.

Another way to convert raw numbers to strings is to use the **String** object directly:

```
> String(100.0);  
'100.0'  
> String(tau);  
'6.283185307179586'
```

We see from the second example that **String** also works on variables.

This method of converting to strings dovetails nicely with going the opposite direction, which uses the **Number** object directly:

```
> Number("6.283185307179586");  
6.283185307179586  
> String(Number("6.283185307179586"));  
'6.283185307179586'  
> Number('1.24e6')  
1240000
```

We see in the final line that JavaScript supports scientific notation:

```
> 1.24e6  
1240000
```

4.1.3 Exercises

1. See if you can guess the return value of `String(Number('1.24e6'))`. Confirm using the Node REPL.
2. Like most programming languages, JavaScript lacks support for imaginary numbers, i.e., numbers that are a real multiple of the *imaginary unit* i (satisfying the equation $i^2 = -1$, sometimes written as $i = \sqrt{-1}$). What is the JavaScript value of the square root of -1 ? By guessing or Googling, figure out what this value stands for. What is its boolean value?

4.2 Dates

Another frequently used built-in object is `Date`, which represents a single moment in time.

The `Date` object gives us our first chance to use the `new` function, a so-called *constructor function* that is the standard JavaScript way to create a new object. So far, we've been able to rely on "literal constructors" like quotes and square brackets, but we can also define things like strings and arrays using `new`:

```
> let s = new String("A man, a plan, a canal-Panama!");
> s;
[String: 'A man, a plan, a canal-Panama!']
> s.split(", ");
[ 'A man', 'a plan', 'a canal-Panama!' ]
```

and

```
> let a = new Array();
> a.push(3);
1
> a.push(4);
2
> a.push("hello, world!");
3
> a;
[ 3, 4, 'hello, world!' ]
> a.pop();
'hello, world!'
```

Unlike strings and arrays, dates have no literal constructor, so we *have* to use **new** in this case:

```
> let now = new Date();  
> now;  
2018-03-01T05:01:47.136Z  
> let moonLanding = new Date("July 20, 1969 20:18");  
> now - moonLanding;  
1534038227136
```

The result here is the number of milliseconds since the day and time of the first Moon landing (Figure 4.2).⁴ (Your results, of course, will vary, because time marches on, and your value for **new Date()** will differ.)

4. Image courtesy of Castleski/Shutterstock.



Figure 4.2: Buzz Aldrin and Neil Armstrong somehow got to the Moon (and back!) without JavaScript.

As with other JavaScript objects, **Date** objects respond to a variety of methods:

```
> now.getFullYear();           // Gives a weird answer
118
> now.getFullYear();           // This is what we want instead.
2018
> now.getMonth();
1
> now.getDay();
3
```

The first line here shows that sometimes the results of JavaScript methods are confusing, so it's important to be wary and double-check the values by hand from time to time.

Things like the month and day are returned as indices, and like everything in JavaScript they are zero-offset. For example, month **0** is January, month **1** is February, month **2** is March, etc.

Even though the official international standard is that Monday is the first day, JavaScript follows the American convention of using Sunday instead. We can get the name of the day by making an array of strings for the days of the week, and then using **getDay()** as an index in the array with the square bracket notation ([Section 3.1](#)):

```
> let daysOfTheWeek = ["Sunday", "Monday", "Tuesday", "Wednesday",
                        "Thursday", "Friday", "Saturday"];
> daysOfTheWeek[now.getDay()];
'Wednesday'
```

Your results will vary, of course, unless you happen to be reading this on a Wednesday.

As a final exercise, let's update our web page with an alert including the day of the week. The code appears in [Listing 4.1](#), with the result as shown in [Figure 4.3](#).

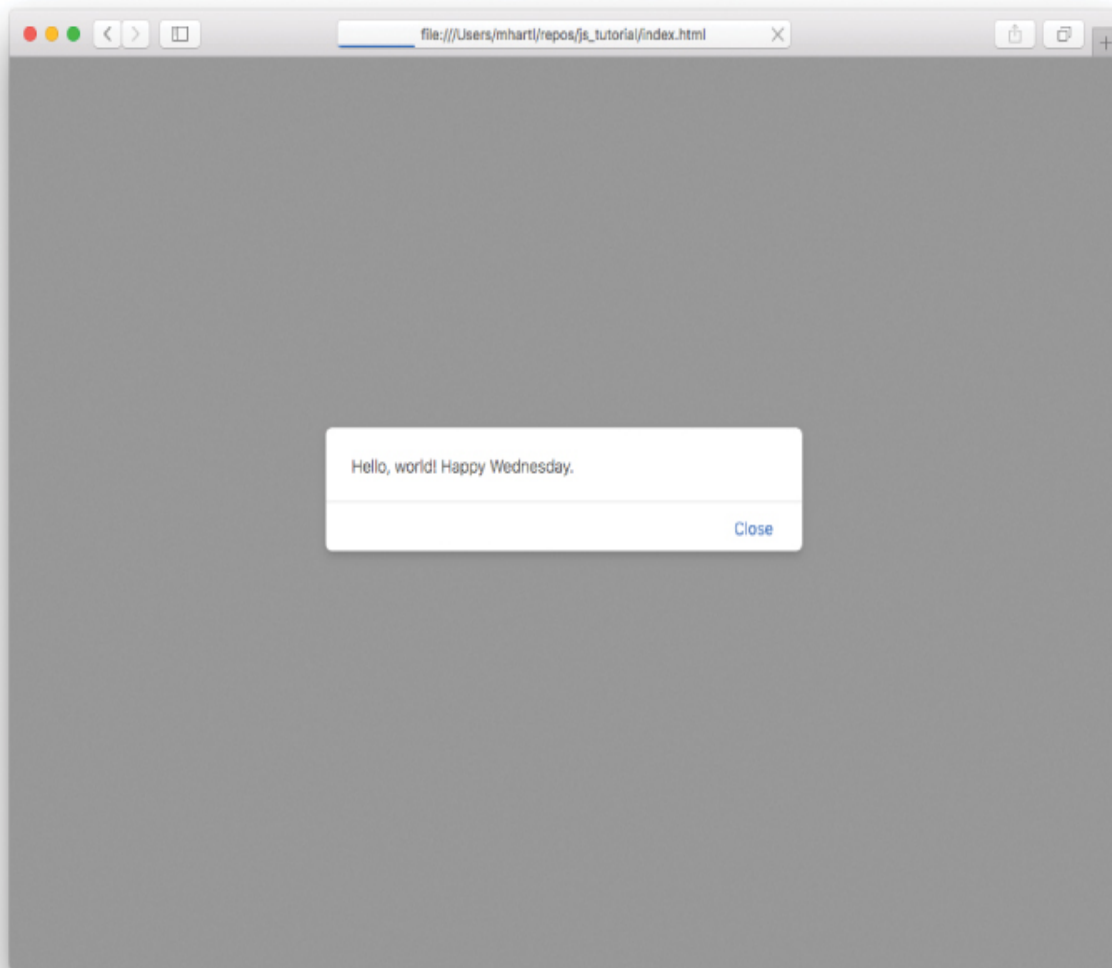


Figure 4.3: A greeting customized just for today.

Listing 4.1: Adding a greeting customized to the day of the week.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
    <script>
      const daysOfTheWeek = ["Sunday", "Monday", "Tuesday",
                             "Wednesday",
                             "Thursday", "Friday", "Saturday"];
      let now = new Date();
      let dayName = daysOfTheWeek[now.getDay()];
      alert(`Hello, world! Happy ${dayName}.`);
    </script>
```

```
</head>
<body>

</body>
</html>
```

Note that [Listing 4.1](#) uses **const** instead of **let** when defining **daysOfTheWeek**:

```
const daysOfTheWeek = ["Sunday", "Monday", "Tuesday", "Wednesday",
                      "Thursday", "Friday", "Saturday"];
```

Here **const**, which (as you can probably guess) is short for “constant”, gives us a way to indicate that the value of the variable won’t change.⁵ Some people even go so far as to use **const** in preference to **let** whenever possible. My preference is to use **let** as a default, and to use **const** as a signal that it’s especially important for the value not to change.

5. Technically, **const** creates an *immutable binding*—i.e., the name can’t change, but the value can. Mutating the contents of a variable created using **const** is a bad practice, though, and should be avoided to prevent confusion.

4.2.1 Exercises

1. Create a new **Date** object by passing it a string for your birthday (including year). JavaScript supports a number of different formats, so it will probably work with whichever date format you prefer. Pretty cool, no?
2. How many seconds after the Moon landing were you born? (Or maybe you were even born *before* the Moon landing—in which case, lucky you! I hope you got to watch it on TV.)

4.3 Regular Expressions

JavaScript has full support for *regular expressions*, often called *regexes* or *regexps* for short, which are a powerful mini-language for matching patterns in text. A full mastery of regular expressions is beyond the scope of this book (and perhaps beyond the scope of human ability), but the good news is that there are many resources available for learning about them incrementally.

(Some such resources are mentioned in “Grepping” (https://www.learnenough.com/command-line-tutorial/inspecting_files) in *Learn Enough Command Line to Be Dangerous* (<https://www.learnenough.com/command-line>) and “Global find and replace” (https://www.learnenough.com/text-editor-tutorial/advanced_text_editing#sec-global_find_and_replace) in *Learn Enough Text Editor to Be Dangerous* (<https://www.learnenough.com/text-editor>).) The most important thing to know about is the general idea of regular expressions; you can fill in the details as you go along.

Regexes are notoriously terse and error-prone; as programmer Jamie Zawinski famously said:

Some people, when confronted with a problem, think “I know, I’ll use regular expressions.” Now they have two problems.

Luckily, this situation is greatly ameliorated by web applications like regex101 (<https://regex101.com/>), which let us build up regexes interactively (Figure 4.4). Moreover, such resources typically include a quick reference to assist us in finding the code for matching particular patterns (Figure 4.5).

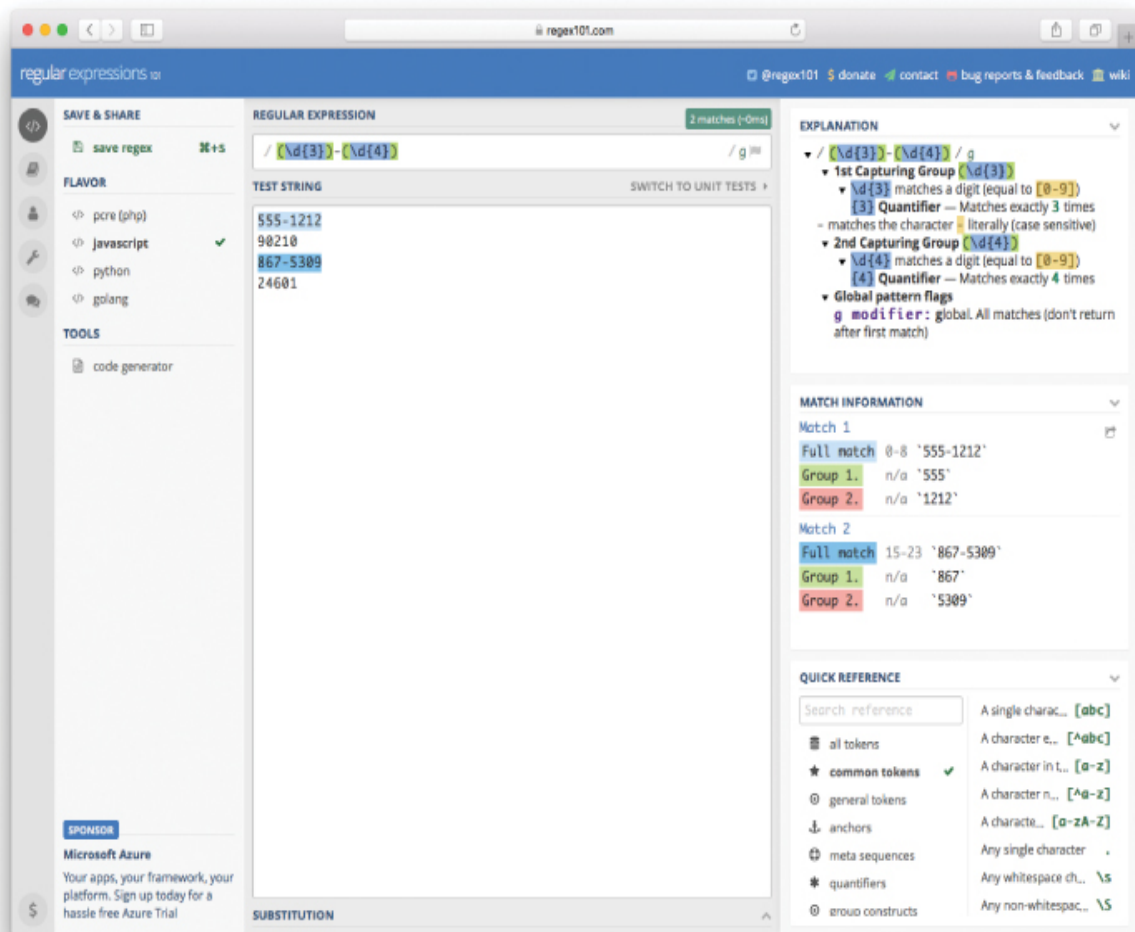


Figure 4.4: An online regex builder.

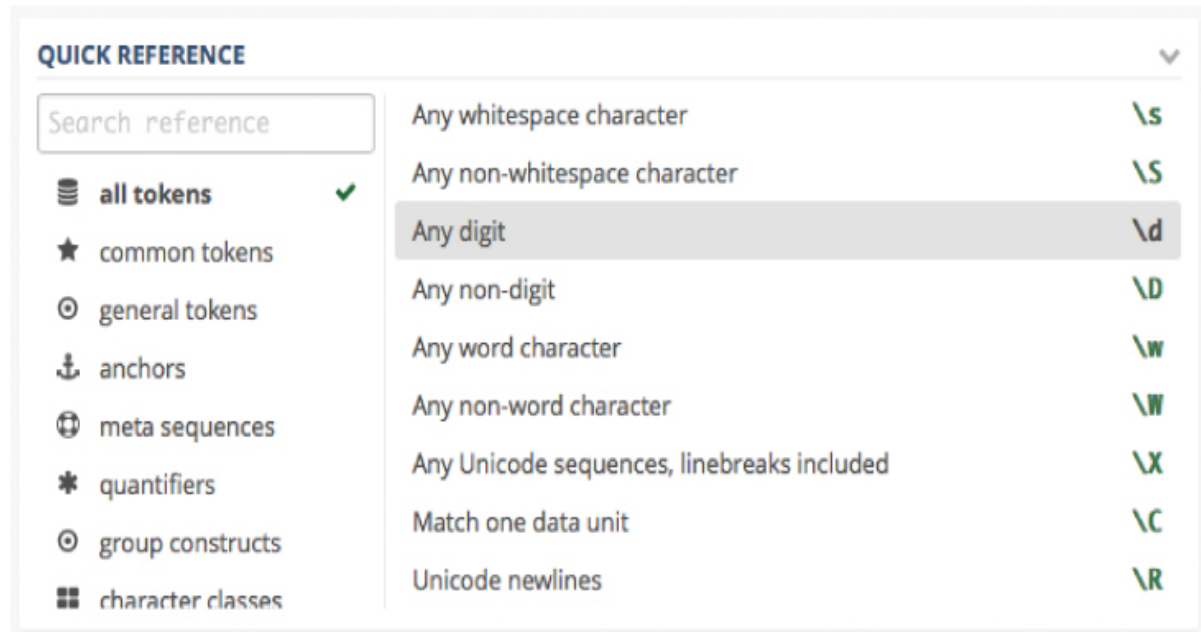


Figure 4.5: A close-up of the regex reference.

If you look carefully at [Figure 4.4](#), you might be able to see the checkmark in the menu on the left indicating that “javascript” has been selected for the regex input format. This arranges to use the exact regular expression conventions we need in this tutorial. In practice, languages differ little in their implementation of regular expressions, but it’s wise to use the correct language-specific settings, and always to double-check when moving a regex to a different language.

Let’s take a look at some simple regex matches in JavaScript. Our examples will draw on both the regex methods and string methods specialized for regexes. (The latter are often more convenient in practice.)

4.3.1 Regex Methods

A basic regex consists of a sequence of characters that matches a particular pattern. We can create a new regex using the **new** function ([Section 4.2](#)) on the **RegExp** object. For example, here’s a regex that matches standard American ZIP codes ([Figure 4.6](#)),⁶ consisting of five digits in a row:

⁶ Image courtesy of 4kclips/123RF.

```
> let zipCode = new RegExp("\\d{5}");
```



Figure 4.6: 90210 is one of the most expensive ZIP codes in America.

Here `\d` represents any digit (0–9), and the first backslash is needed to *escape* the second backslash to get a literal backslash in the string. (We’ll see how to avoid this inconvenient requirement using a literal regex constructor in [Section 4.3.2](#).) Meanwhile, `{5}` says to match exactly five digits in a row.

If you use regular expressions a lot, eventually you’ll memorize many of these rules, but you can always look them up in a quick reference ([Figure 4.5](#)).

Now let’s see how to tell if a string matches a regex. Regular expressions come with an `exec` method that “executes” the regex on a string:

```
> let result = zipCode.exec("Beverly Hills 90210");  
> result;  
[ '90210', index: 14, input: 'Beverly Hills 90210' ]
```

The result here includes the matching string, the index number where the match starts, and the original input.

I don't like the format of the result above, mainly because the output is a weird and confusing pseudo-array that seemingly has three elements but in fact has length **1**:

```
> result.length  
1
```

4.3.2 String Methods

A more convenient way to make regex matches is to use string methods. There's also a literal regex constructor syntax that's more convenient for most purposes.

As we learned in [Section 4.2](#), some JavaScript objects need to be created using **new**, while others have optional literal constructors, such as quotes for making strings and square brackets for making arrays. Regexes support just such a literal constructor syntax, namely, patterns inside of forward slashes:

```
> zipCode = /\d{5}/;  
/\d{5}/
```

Note that, unlike in the case of the named **RegExp** constructor in [Section 4.3.1](#), when using the literal constructor we don't have to escape the **\d** with an extra backslash.

Now let's build up a longer string with multiple ZIP codes ([Figure 4.7](#)):⁷

⁷. Image courtesy of kitleong/123RF.



Figure 4.7: 91125 is a dedicated ZIP code for the campus of the California Institute of Technology (Caltech).

```
> let s = "Beverly Hills 90210 was a '90s TV show set in Los Angeles.";
> s += " 91125 is another ZIP code in the Los Angeles area."
'Beverly Hills 90210 was a \'90s TV show set in Los Angeles. 91125 is another ZIP code in the Los Angeles area.'
```

You should be able to use your technical sophistication ([Box 1.1](#)) to infer what the `+=` operator does here if you haven't seen it before (which might involve doing a quick Google search).

To find out whether the string matches the regex, we can use the string `match` method:

```
> s.match(zipCode);
[ '90210',
  index: 14,
  input: 'Beverly Hills 90210 was a \'90s TV show set in Los Angeles.
91125 is
        another ZIP code in the Los Angeles area.' ]
```

The result is the same weird pseudo-array we saw in [Section 4.3.1](#), but at least it gives the same result when run a second time:

```
> s.match(zipCode);
[ '90210',
  index: 14,
  input: 'Beverly Hills 90210 was a \'90s TV show set in Los Angeles.
91125 is
        another ZIP code in the Los Angeles area.' ]
```

The **match** method is especially useful in conditionals; recalling the “bang bang” notation from [Section 2.4.2](#), we can evaluate the match in a boolean context:

```
> !!s.match(zipCode);
true
```

Thus, we can do things like this:

```
> if (s.match(zipCode)) {
    "Looks like there's at least one ZIP code in the string!";
}
'Looks like there\'s at least one ZIP code in the string!'
```

Even better, there’s a common technique for matching *all* occurrences of a regular expression using the “global flag” **g** after the second slash:

```
> zipCode = /\d{5}/g;           // Use 'g' to set the 'global' flag.
/\d{5}/g
```

The resulting output is pleasantly intuitive:

```
> s.match(zipCode);
[ '90210', '91125' ]
```

The result here is simply an array of the ZIP codes detected in the string, suitable for joining ([Section 3.4.3](#)) or iterating ([Section 3.5](#) and [Section 5.4](#)).

Our final example of regexes combines the power of pattern matching with the **split** method we saw in [Section 3.1](#). In that section, we split on spaces, like this:

```
> "ant bat cat duck".split(" ");  
[ 'ant', 'bat', 'cat', 'duck' ]
```

We can obtain the same result in a more robust way by splitting on *whitespace*, which consists of spaces, tabs (indicated with **\t**), and newlines (indicated with **\n**).

Consulting the quick reference ([Figure 4.5](#)), we find that the regex for whitespace is **\s**, and the way to indicate “one or more” is the plus sign **+**. Thus, we can split on whitespace as follows:

```
> "ant bat cat duck".split(/\s+/);  
[ 'ant', 'bat', 'cat', 'duck' ]
```

The reason this is so nice is that now we can get the same result if the strings are separated by multiple spaces, tabs, newlines, etc.:⁸

8. This pattern is so useful that it’s the default behavior for **split** in some languages (notably Ruby), so that **"ant\tbat cat".split** is **["ant", "bat", "cat"]**.

```
> "ant bat\tcat\nduck".split(/\s+/);  
[ 'ant', 'bat', 'cat', 'duck' ]
```

We also see here the value of the literal constructor: Especially when using short regexes, there’s no need to define an intermediate variable; instead, we can use the literal regex directly.

4.3.3 Exercises

1. Write a regex that matches the extended-format ZIP code consisting of five digits, a hyphen, and a four-digit extension (such as 10118-0110). Confirm that it works using **String#match** and the caption in [Figure 4.8](#).⁹

9. Image courtesy of jordi2r/123RF.



Figure 4.8: ZIP code 10118-0110 (the Empire State Building).

2. Write a regex that splits only on newlines. Such regexes are useful for splitting a block of text into separate lines. In particular, test your regex by pasting the poem in [Listing 4.2](#) into the console and using `sonnet.split(/your regex/)`. What is the length of the resulting array?

Listing 4.2: Some text with newlines.

```
> const sonnet = `Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove.
O no, it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wand'ring bark,
Whose worth's unknown, although his height be taken.
Love's not time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.`
```

```
If this be error and upon me proved,  
I never writ, nor no man ever loved.`;
```

4.4 Plain Objects

The word *object* is used in various contexts in JavaScript, usually referring to the abstract idea of a collection of data (properties) and functions (methods). As noted in [Section 2.4](#), (almost) everything in JavaScript is an object, and we'll see in Chapter 7 how to define objects that parallel built-in objects like **String**, **Array**, and **RegExp**. In this section, we'll focus on *plain objects*, which are simpler to define than the more general objects we've encountered so far.

In general, objects in JavaScript can be dizzyingly complex, but in their simplest incarnation they work much like hashes (also called *associative arrays*) in other languages. You can think of them as being like regular arrays but with strings rather than integers as indices. Each element is thus a pair of values: a string (the *key*) and an element of any type (the *value*). These elements are also known as *key/value pairs*.

As a simple example, let's create an object to store the first and last names of a user, such as we might have in a web application:

```
> let user = {};           // {} is an empty Object.  
{}  
> user["firstName"] = "Michael"; // Key "firstName", value  
'Michael'  
> user["lastName"] = "Hartl";   // Key "lastName", value "Hartl"  
'Hartl'
```

As you can see, an empty **Object** is represented by curly braces, and we can assign values using the same square bracket syntax as for arrays. We can retrieve values in the same way:

```
> user["firstName"]; // Element access is like arrays  
'Michael'  
> user["lastName"];  
'Hartl'
```

The keys in our object are nothing other than the *properties* we first met in [Section 2.4](#), and as such they can also be accessed using the dot notation we saw with, e.g., **string.length**:

```
> user.firstName;      // Element access using the dot notation
'Michael'
> user.lastName;
'Hartl'
```

Deciding which syntax to use is a matter of context and style. Note that in either case an undefined key/property name simply returns **undefined**:

```
> user["dude"];
undefined
> user.dude;
undefined
> !!user.dude
false
```

The last line here is a reminder that **undefined** is **false** in a boolean context, which you may recall if you solved the corresponding exercise in [Section 3.2.1](#).

Finally, we can simply display or define the full object, thereby showing the key/value pairs ([Listing 4.3](#)).

Listing 4.3: A literal representation of an object.

```
> user;
{ firstName: 'Michael', lastName: 'Hartl' }
> let otherUser = { firstName: 'Foo', lastName: 'Bar' };
> otherUser["firstName"];
'Foo'
> otherUser["lastName"];
'Bar'
```

4.4.1 Exercises

1. Show that **new Object()** also works to create a new empty object. What happens if you give the object constructor an argument equal to the output of [Listing 4.3](#)?

4.5 Application: Unique Words

Let's apply plain objects to a challenging exercise, consisting of our longest program so far. Our task is to extract all of the unique words in a fairly long piece of text, and count how many times each word appears.

Because the sequence of commands is rather extensive, our main tool will be a JavaScript file ([Section 1.4](#)), executed using the **node** command. (We're not going to make it a self-contained shell script as in [Section 1.5](#) because we don't intend this to be a general-purpose utility program.) At each stage, I suggest using a Node REPL to execute the code interactively if you have any question about the effects of a given command.

Let's start by creating our file:

```
$ touch count.js
```

Now fill it with a **const** containing the text, which we'll choose to be Shakespeare's Sonnet 116¹⁰ ([Figure 4.9](#)),¹¹ as borrowed from [Listing 4.2](#) and shown again in [Listing 4.4](#).

¹⁰. Note that in the original pronunciation used in Shakespeare's time, words like "love" and "remove" rhymed, as did "come" and "doom".

¹¹. Image courtesy of psychoshadowmaker/123RF.



Figure 4.9: Sonnet 116 compares love’s constancy to the guide star for a wandering bark (ship).

Listing 4.4: Adding some text as a **const**.
count.js

```
const sonnet = `Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove.
O no, it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wand'ring bark,
Whose worth's unknown, although his height be taken.
Love's not time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
    If this be error and upon me proved,
    I never writ, nor no man ever loved.`;
```

Note that [Listing 4.4](#) uses the backtick syntax ([Section 2.2.1](#)), which (as it turns out) allows us to break text across lines (unlike regular quotes).¹² Note:

Because this syntax is relatively new, some text editors (notably some versions of Sublime Text) might need to be configured to highlight it properly.

12. I had to Google around to learn how to do this.

Next, we'll initialize our object, which we'll call **uniques** because it will have an entry for each unique word in the text:

```
let uniques = {};
```

For the purposes of this exercise, we'll define a “word” as a run of one or more *word characters* (i.e., letters or numbers, though there are none of the latter in the present text). This match can be accomplished with a regular expression ([Section 4.3](#)), which includes a pattern (`\w`) for exactly this case ([Figure 4.5](#)):

```
let words = sonnet.match(/\w+/g);
```

This uses the “global” **g** flag and the **match** method from [Section 4.3.2](#) to return an array of all the strings that match “one or more word characters in a row”. (Extending this pattern to include apostrophes (so that it matches, e.g., “wand’ring” as well) is left as an exercise ([Section 4.5.2](#)).

At this point, the file should look like [Listing 4.5](#).

Listing 4.5: Adding an object and the matching words.
count.js

```
const sonnet = `Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove.
O no, it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wand'ring bark,
Whose worth's unknown, although his height be taken.
Love's not time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
    If this be error and upon me proved,
    I never writ, nor no man ever loved.`;
```

```
let uniques = {};  
let words = sonnet.match(/\w+/g);
```

Now for the heart of our program. We're going to loop through the **words** array ([Section 3.5](#)) and do the following:

1. If the word already has an entry in the **uniques** object, increment its count by **1**.
2. If the word doesn't have an entry yet in **uniques**, initialize it to **1**.

The result, using the **+=** operator we met briefly in [Section 4.3.2](#), looks like this:

```
for (let i = 0; i < words.length; i++) {  
  let word = words[i];  
  if (uniques[word]) {  
    uniques[word] += 1;  
  } else {  
    uniques[word] = 1;  
  }  
}
```

Among other things, we see here the power of the bracket access notation, as there would be no way to accomplish this same task using the dot syntax. Note also that we're relying on **uniques[word]** being undefined (**false** in a boolean context) if **word** isn't yet a valid key.

Finally, we'll print out the result to the terminal:

```
console.log(uniques)
```

The full program (with added comments) appears as in [Listing 4.6](#).

Listing 4.6: A program to count words in text.
count.js

```
const sonnet = `Let me not to the marriage of true minds  
Admit impediments. Love is not love  
Which alters when it alteration finds,  
Or bends with the remover to remove.  
O no, it is an ever-fixed mark  
That looks on tempests and is never shaken;
```

```
It is the star to every wand'ring bark,  
Whose worth's unknown, although his height be taken.  
Love's not time's fool, though rosy lips and cheeks  
Within his bending sickle's compass come:  
Love alters not with his brief hours and weeks,  
But bears it out even to the edge of doom.  
    If this be error and upon me proved,  
    I never writ, nor no man ever loved.`;
```

```
// Unique words  
let uniques = {};  
// All words in the text  
let words = sonnet.match(/\w+/g);  
// Iterate through `words` and build up an associative array of  
// unique words.  
for (let i = 0; i < words.length; i++) {  
    let word = words[i];  
    if (uniques[word]) {  
        uniques[word] += 1;  
    } else {  
        uniques[word] = 1;  
    }  
}  
  
console.log(uniques)
```

It's worth noting that, even in a relatively short program like [Listing 4.6](#), it can be tricky to get all the braces, parentheses, etc., to match up. A good text editor can help; for example, when the cursor is next to a closing curly brace, Atom displays subtle bars under each member of the opening/closing pair ([Figure 4.10](#)).



Figure 4.10: Text editors can help immensely in matching up curly braces, etc.

The result of running `count.js` in the terminal looks something like this:

```
$ node count.js
{ Let: 1,
  me: 2,
  not: 4,
  to: 4,
  the: 4,
  marriage: 1,
  .
  .
  .
  upon: 1,
  proved: 1,
  I: 1,
  writ: 1,
  nor: 1,
  man: 1,
  loved: 1 }
```

4.5.1 Map

Although native JavaScript objects can be used as hashes/associative arrays (as we've done above), they do have their weaknesses, such as slower performance and limited support for extracting keys and values. JavaScript comes with a dedicated **Map** object to address these limitations, with **set** and **get** methods for setting and getting values using keys:

```
> let uniques = new Map();
> uniques.set("loved", 0);
Map { 'loved' => 0 }
> let currentValue = uniques.get("loved");
> uniques.set("loved", currentValue + 1);
Map { 'loved' => 1 }
```

Combining the techniques shown above to rewrite the `count.js` program is left as an exercise ([Section 4.5.2](#)).

4.5.2 Exercises

1. Extend the regex used in [Listing 4.6](#) to include an apostrophe, so it matches, e.g., “wand’ring”. *Hint:* Combine the first reference regex at regex101 ([Figure 4.11](#)) with `\w`, an apostrophe, and the plus operator `+`.

QUICK REFERENCE			▼
Search reference			
☰ all tokens	A character in the range: a-z	[a-z]	
★ common tokens ✓	A character not in the range: a-z	[^a-z]	
⊙ general tokens	A character in the range: a-z or A-Z	[a-zA-Z]	
⚓ anchors	Any single character	.	
⊕ meta sequences	Any whitespace character	\s	
* quantifiers	Any non-whitespace character	\S	
⊙ group constructs	Any digit	\d	
-- . . .	Any non-digit	\D	
	Any word character	\w	

Figure 4.11: An exercise hint.

2. Rewrite [Listing 4.6](#) using **Map** (Section 6.1) instead of native JavaScript objects.

Chapter 5. Functions

So far in this tutorial, we've repeatedly mentioned JavaScript functions, and in this chapter we'll finally learn how to define functions of our own. The resulting ability gives us greater flexibility as programmers, and enables powerful techniques like **forEach** (Section 5.4) and *functional programming* (Chapter 6). Functions: achievement unlocked (Figure 5.1).



Figure 5.1: Time to level up.

5.1 Function Definitions

As we saw in Section 1.2, function calls in JavaScript consist of a *name* and zero or more arguments enclosed in parentheses:

```
> console.log("hello, world!");  
hello, world!
```

As discussed in Section 2.5, functions attached to objects (such as **log** attached to **console**) are also called *methods*.

One of the most important tasks in programming involves defining our own functions. Let's take a look at a simple example in the REPL. We'll define a function called **stringMessage** that takes a single argument and returns a message based on whether the argument is empty or not:

```
> function stringMessage(string) {  
    if (string) {
```

```
    return "The string is nonempty.";
  } else {
    return "It's an empty string!";
  }
}
undefined
```

Note the use of **return** to indicate the *return value* of the function. The result can be seen by calling the function in the REPL:

```
> stringMessage("honey badger");
'The string is nonempty.'
> stringMessage("");
'It\'s an empty string!'
```

It's important to understand that the name of the function argument is irrelevant as far as the caller is concerned. In other words, the first example above could replace **string** with any other valid variable name, such as **asdf**, and it would work just the same:

```
> function stringMessage(asdf) {
  if (asdf) {
    return "The string is nonempty.";
  } else {
    return "It's an empty string!";
  }
}
undefined
> stringMessage("honey badger");
'The string is nonempty.'
> stringMessage("");
'It\'s an empty string!'
```

5.1.1 Sorting Numerical Arrays

We can apply functions to solve the conundrum encountered in [Section 3.4.1](#), where we saw that JavaScript sorts even numerical arrays “alphabetically” by default:

```
> let a = [8, 17, 42, 99];
> a.sort();
[ 17, 42, 8, 99 ]
```

To sort an array numerically, we can define a function (which we'll call **number-Compare**) that takes in two numbers **a** and **b** and returns **1** when **a > b**, **-1** when **a < b**, and **0** when they're equal. This is the form required by the Array sort documentation (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort), and as we'll see it will let **sort** figure out that we want to sort the array numerically rather than alphabetically. The result appears in [Listing 5.1](#).

Listing 5.1: Comparing numbers numerically.

```
> function numberCompare(a, b) {  
  if (a > b) {  
    return 1;  
  } else if (a < b) {  
    return -1;  
  }  
  else {  
    return 0;  
  }  
}
```

We can test this directly as follows:

```
> numberCompare(8, 17);  
-1  
> numberCompare(17, 99);  
-1  
> numberCompare(99, 42);  
1  
> numberCompare(99, 99);  
0
```

At this point, we can sort the array by giving **sort** a *function argument*, which causes it to change its default comparison:

```
> a.sort(numberCompare);  
[ 8, 17, 42, 99]
```

This is the result we're looking for. Under the hood, JavaScript is marching through the array and asking the question, "What is the value of **numberCompare(a, b)**? If it's negative, **a** goes before **b**; if it's positive, **b** goes before **a**. (If they're equal, it doesn't matter.)" The result is an array sorted according to what we intuitively expect.

We'll see an even better way to sort arrays when we learn about *anonymous functions* (functions without a name) in [Section 5.4](#). In particular, an exercise in [Section 5.4.1](#) includes the most idiomatically correct way to sort JavaScript arrays numerically ([Listing 5.17](#)).

5.1.2 Fat Arrow

ES6 adds a second method for defining functions known as a “fat arrow”, indicated with an equals sign followed by a right angle bracket: `=>`. We can define an alternate string message function by combining `=>` and the `let` keyword:

```
> let altStringMessage = (string) => {  
  if (string) {  
    return "The string is nonempty.";  
  } else {  
    return "It's an empty string!";  
  }  
}  
> altStringMessage("honey badger");  
'The string is nonempty '
```

Here `(string) => ...` says to create a function with one argument (`string`) defined by the code to the right of the arrow. In other words,

```
function name(arg) {  
  // code  
}
```

is the same as

```
let name = (arg) => {  
  // code  
}
```

Some developers prefer to use this alternate notation for *all* their functions, and this may eventually be a universal practice, but for now the use of `function` is very common. It also maps more cleanly to the way you actually call functions:

```
function foo(bar, baz) {  
  // do something with bar and baz  
}  
  
let x = 1;  
let y = 2;  
let result = foo(x, y);
```

Because the actual code here uses **foo(x, y)**, it's nice to see a similar sequence of characters appear in the function definition.

In this tutorial, we'll generally stick to using **function** to define functions, but in some contexts we will use the arrow notation for anonymous functions, especially in the context of the *functional programming* techniques discussed in Chapter 6.

As with **var** and **let/const**, both **function** and **=>** are in common use. Even if you standardize on one convention for your own code, you have to know both syntaxes in order to read code written by others.

5.1.3 Exercises

1. Using **function**, define a **square** function that returns the square of a number. Do the same with an analogous **altSquare** function using the fat arrow notation.

5.2 Functions in a File

Although defining functions in a REPL is convenient for demonstration purposes, it's a bit cumbersome, and a better practice is to put them in a file (as we did with the script in [Section 4.5](#)). We'll start by defining a function directly in the **index.html** file created in [Section 1.2](#), and will then move the function to an even more convenient external file.

Recall from [Section 4.2](#) that **Date** objects have a **getDay()** method that returns the index corresponding to the day of the week (**0** for Sunday, **1** for Monday, etc.). In [Listing 4.1](#), we defined a **const** for the days of the week, and then defined a **dayName** variable using **getDay()**:

```
const daysOfTheWeek = ["Sunday", "Monday", "Tuesday", "Wednesday",  
                      "Thursday", "Friday", "Saturday"];  
let now = new Date();  
let dayName = daysOfTheWeek[now.getDay()];  
alert(`Hello, world! Happy ${dayName}.`);
```

It would be convenient to *encapsulate* this definition and logic in a **dayName** function, so that we could write our alert like this:

```
alert(`Hello, world! Happy ${dayName(now)}.`);
```

This eliminates the use of the **dayName** variable, instead replacing it with the function call **dayName(now)**.

Applying the function definition syntax from [Section 5.1](#) leads to the following **dayName** function:

```
function dayName(date) {  
    const daysOfTheWeek = ["Sunday", "Monday", "Tuesday", "Wednesday",  
                          "Thursday", "Friday", "Saturday"];  
    return daysOfTheWeek[date.getDay()];  
}
```

Note how we've switched from the variable **now** to the more generic-sounding **date**, which is a way of indicating that our function works with any date that's passed to it.

Putting this definition into the code from [Listing 4.1](#) nicely separates the **alert** logic from the code used to generate the day of the week, as shown in [Listing 5.2](#). The result appears in [Figure 5.2](#).

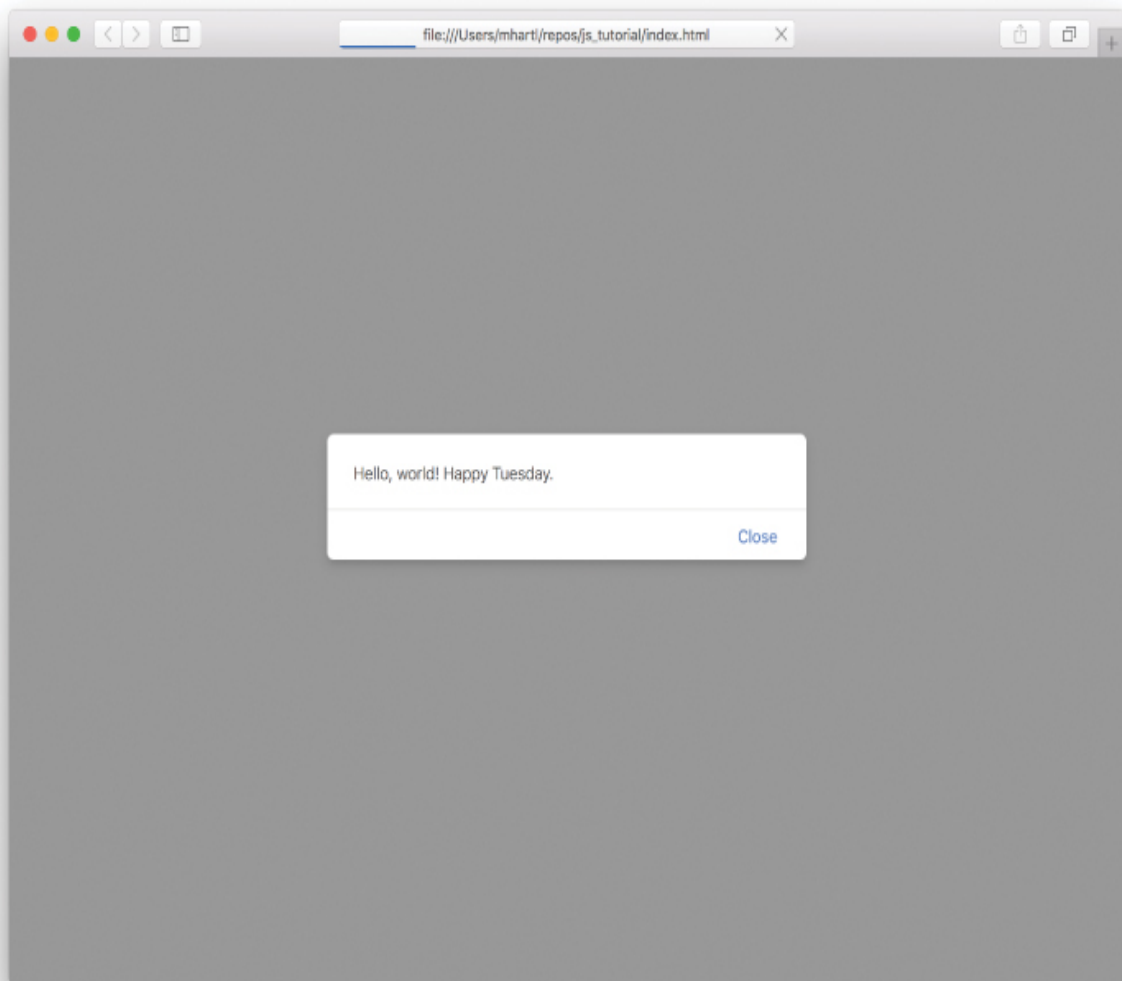


Figure 5.2: The result of a functional greeting.

Listing 5.2: Factoring the day of the week into a function.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
    <script>
      function dayName(date) {
        const daysOfTheWeek = ["Sunday", "Monday", "Tuesday",
"Wednesday",
                                "Thursday", "Friday",
"Saturday"];
        return daysOfTheWeek[date.getDay()];
```

```
    }

    let now = new Date();
    alert(`Hello, world! Happy ${dayName(now)}.`);
  </script>
</head>
<body>

</body>
</html>
```

We can make the code in [Listing 5.2](#) even cleaner by factoring the **dayName** function into a separate file and then including it into our page. We'll start by cutting the function and pasting it into a new file, **day.js**:

```
$ touch day.js
```

The resulting files appear as in [Listing 5.3](#) and [Listing 5.4](#).¹

1. In some editors, you can use Shift-Command-V to paste in a selection using the local indentation level, which saves us the trouble of dedenting it by hand.

Listing 5.3: The **dayName** function in a file.
day.js

```
// Returns the day of the week for the given date.
function dayName(date) {
  const daysOfTheWeek = ["Sunday", "Monday", "Tuesday",
    "Wednesday",
    "Thursday", "Friday", "Saturday"];
  return daysOfTheWeek[date.getDay()];
}
```

Listing 5.4: Our greeting with a function in a file.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
    <script>
      let now = new Date();
      alert(`Hello, world! Happy ${dayName(now)}!`);
    </script>
```

```
</head>
<body>

</body>
</html>
```

As you can verify by reloading the browser, at this point our `index.html` page is simply blank—there’s no alert, and the JavaScript doesn’t work. The default behavior is simply a silent error, but using the browser console ([Section 1.3.1](#)) shows the problem, as seen in [Figure 5.3](#). Using the console in this manner is a powerful debugging technique—if your JavaScript ever just silently fails, firing up the browser console should be your strategy of first resort ([Box 5.1](#)).²

2. @ThePracticalDev image used with permission.

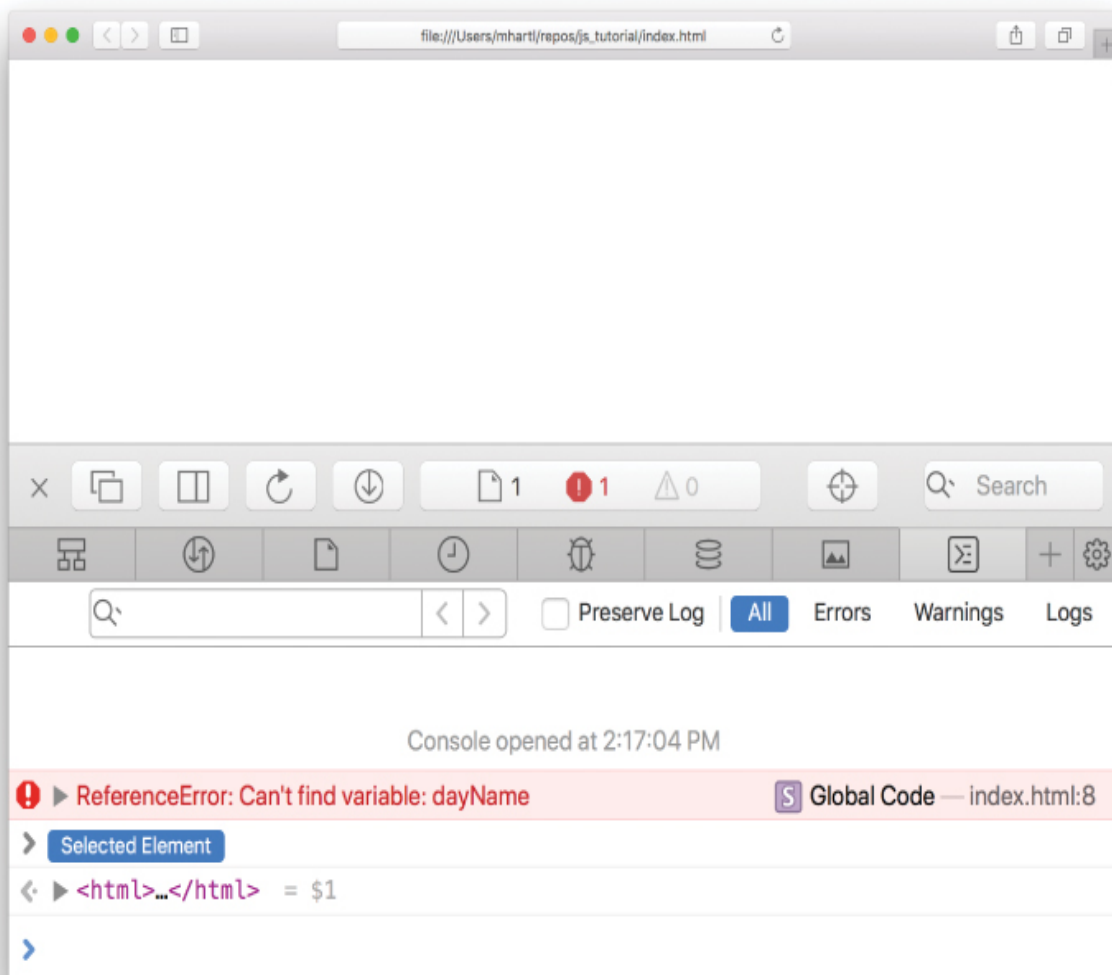


Figure 5.3: Using the browser console to find an error.

Box 5.1: Debugging JavaScript

One skill that's an essential part of technical sophistication is *debugging*: the art of finding and correcting errors in computer programs. While there's no substitute for experience, here are some techniques that should give you a leg up when tracking down the inevitable glitches in your code:

- *Fire up the browser console.* Often, this alone will allow you to identify the bug, especially when the error is silent (i.e., when the program just fails in the browser without any indication of why).
- *Trace the execution with logs or alerts.* When trying to figure out why a particular program is going awry, it's often helpful to display variable values with temporary `console.log` or `alert` statements, which can be removed when the bug is fixed.
- *Comment out code.* It's sometimes a good idea to comment out code you suspect is unrelated to the problem to allow you to focus on the code that isn't working.
- *Use the REPL.* Firing up the REPL and pasting in the problematic code is frequently an excellent way to isolate the problem.
- *Google it.* Googling error messages or other search terms related to the bug (which often leads to helpful threads at Stack Overflow) is an essential skill for every modern software developer ([Figure 5.4](#)).

The internet will make those bad words go away



Essential

Googling the Error Message

O RLY?

The Practical Developer
@ThePracticalDev

Figure 5.4: How did people ever debug before Google?

The problem is that we've removed **dayName** from the **script** section of **index.html**, so naturally our page has no idea what it is. The solution is to include it using a second script tag with the **src** (source) attribute pointing to the file containing the definition:

```
<script src="day.js"></script>

<script>
  let now = new Date();
  alert(`Hello, world! Happy ${dayName(now)}!`);
</script>
```

This code might look familiar, since it is similar to the syntax for including images (https://www.learnenough.com/html-tutorial/filling_in_the_index_page#sec-images) (Figure 5.5):³

3. Image courtesy of halfmax.ru/Shutterstock.



Figure 5.5: Recognize this cutie (<https://www.learnenough.com/html-tutorial#fig-kitten>) from *Learn Enough HTML to Be Dangerous* (<https://www.learnenough.com/html>)?

```

```

In particular, **src** needs to have the full path to the file, so a JavaScript source file **site.js** in a **scripts/** directory would be referenced using

```
<script src="scripts/site.js"></script>
```

The result of putting our new **script** tag with **src** into **index.html** appears in Listing 5.5. Upon reloading the page, our greeting now appears as expected (Figure 5.6).

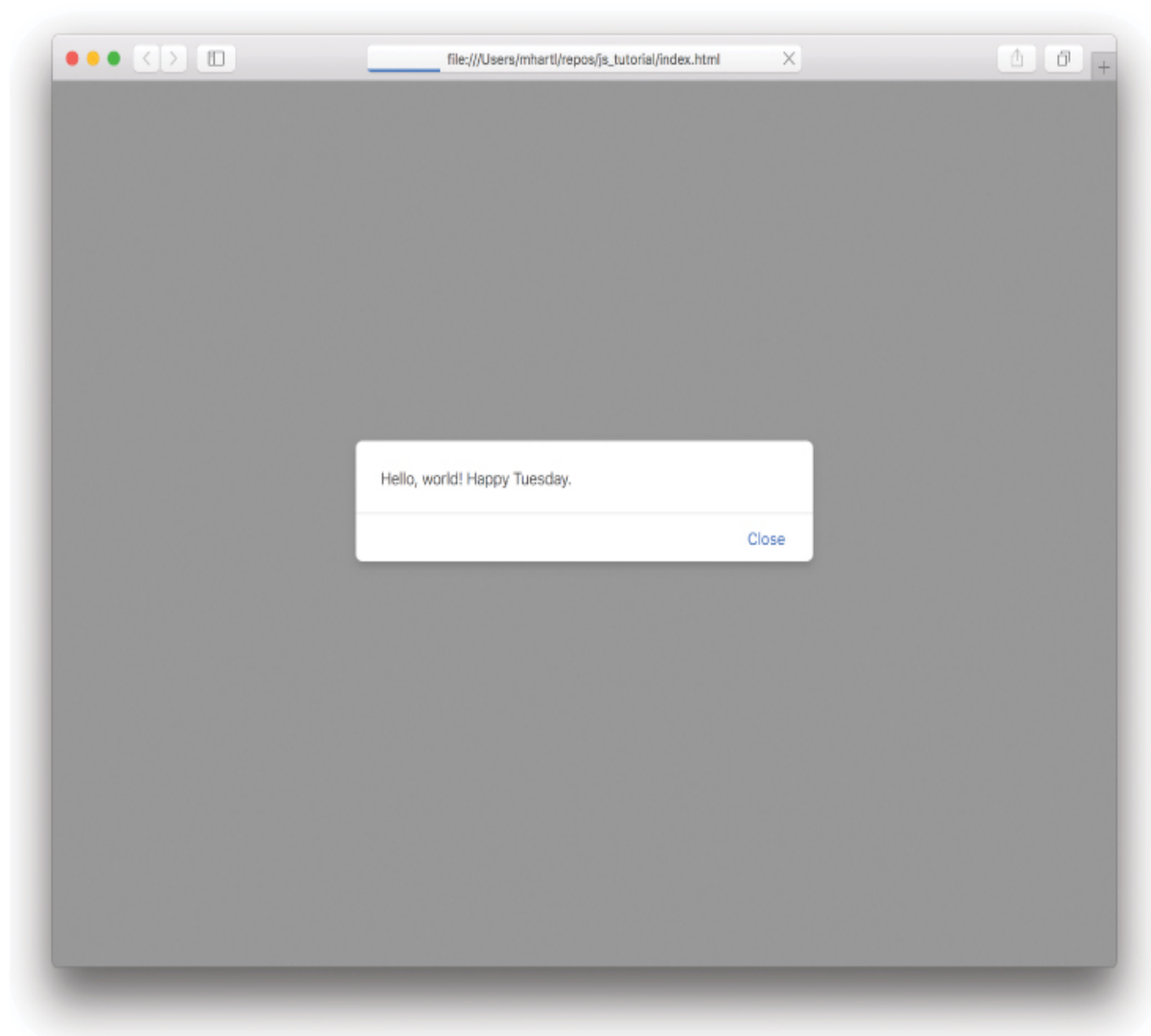


Figure 5.6: The greeting restored.

Listing 5.5: Using a function from an external file.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
    <script src="day.js"></script>
    <script>
      let now = new Date();
      alert(`Hello, world! Happy ${dayName(now)}!`);
    </script>
  </head>
  <body>
```

```
</body>
</html>
```

It's important to note that the source script has to come *before* any functions defined in the script are used. Otherwise, the result is the same as in [Figure 5.3](#). Confirming this is left as an exercise ([Section 5.2.1](#)).

5.2.1 Exercises

1. What happens if the **src** line in [Listing 5.5](#) comes after the main script? What is the error in the console?
2. Let's replace the interpolated string in [Listing 5.4](#) with a **greeting** function in **day.js**. Fill in the code in [Listing 5.6](#) to get [Listing 5.7](#) to work.

Listing 5.6: Defining a **greeting** function.
day.js

```
// Returns the day of the week for the given date.
function dayName(date) {
    const daysOfTheWeek = ["Sunday", "Monday", "Tuesday",
                           "Wednesday", "Thursday", "Friday", "Saturday"];
    return daysOfTheWeek[date.getDay()];
}

// Returns a greeting for the given date.
function greeting(date) {
    // FILL IN
}
```

Listing 5.7: Using the **greeting** function.
index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Learn Enough JavaScript</title>
    <meta charset="utf-8">
    <script src="day.js"></script>
    <script>
      let now = new Date();
      alert(greeting(now));
    </script>
  </head>
</html>
```

```
</script>
</head>
<body>

</body>
</html>
```

5.3 Method Chaining

In this section, we'll start developing the palindrome theme mentioned in the introduction ([Chapter 1](#)). Our goal is to write a function called **palindrome** that returns **true** if its argument is the same forward and backward, and **false** otherwise.

We can express the simplest possible definition of a palindrome as “a string and the string reversed are the same.” (We'll steadily expand this definition over time.) In code, we can write this definition as follows:

```
function palindrome(string) {
  return string === reverse(string);
}
```

As required, this will return **true** if a string is a palindrome (equal to its own reverse), and **false** otherwise. There's just one problem, though: JavaScript has no native way to reverse a string, so **reverse(string)** in our proposed implementation won't work. This means we'll have to write the **reverse** function ourselves.

Our technique involves a useful practice called *method chaining*, whereby we call a series of methods one after the other. In particular, although JavaScript doesn't have a native way to reverse strings, we saw in [Section 3.4.1](#) that it *does* have a native way to reverse *arrays*:

```
> let a = [ 17, 42, 8, 99 ];
> a.reverse();
[ 99, 8, 42, 17 ]
```

Meanwhile, we saw in [Section 3.1](#) how to decompose a string into its constituent characters by splitting on the empty string `""`:

```
> "racecar".split("");  
[ 'r', 'a', 'c', 'e', 'c', 'a', 'r' ]
```

Finally, we learned in [Section 3.4.3](#) that the **join** method effectively undoes a split:

```
> [ 'r', 'a', 'c', 'e', 'c', 'a', 'r' ].join("");  
'racecar'
```

This discussion suggests the following algorithm for writing a **reverse** method:

1. Split a string on the empty string to create an array of characters.
2. Reverse the array.
3. Join the array on the empty string to create the reversed string.

Because of method chaining, we can implement this algorithm in a single line:

```
> let string = "Racecar";  
> string.split("").reverse().join("")  
'racecaR'
```

(Because of the way **split("")** works, this method will actually fail for text containing more complex characters like emojis. We'll fix this minor blemish in [Section 5.3.1](#).)

Let's put the **reverse** function into a library for detecting palindromes, which we'll call **palindrome.js**:

```
$ touch palindrome.js
```

The resulting function appears in [Listing 5.8](#).

Listing 5.8: A function for reversing a string.
palindrome.js

```
// Reverses a string.  
function reverse(string) {  
  return string.split("").reverse().join("");  
}
```

Note that [Listing 5.8](#) includes a *documentation comment* (introduced briefly in [Listing 5.3](#)) explaining the purpose of the function. This isn't strictly required, but it's an excellent practice for the sake of future programmers (including us!).

To check the effect of [Listing 5.8](#), we can **load** the external file in a Node REPL using “dot load” (note the lack of an ending semicolon, which is necessary to avoid an error):

```
> .load palindrome.js
> reverse("Racecar");
'racecaR'
```

(Using **.load** simply runs every line of the file inside the REPL. This has the side effect of altering our command history, so that using “up arrow” to retrieve previous commands is less useful than it would be otherwise. There's really no way around this; unfortunately, as a consequence of its web-browser origins, JavaScript doesn't have a native way to include one file into another, so the **.load** kluge is the best we can do.)

We are now in a position to write the first version of our palindrome function, which will compare a string to its own reverse:

```
> string;      // Just a reminder of what our string is
'Racecar'
> string === reverse(string);
false
```

The resulting definition of **palindrome** appears in [Listing 5.9](#).

Listing 5.9: Our initial **palindrome** function.
palindrome.js

```
// Reverses a string.
function reverse(string) {
  return string.split("").reverse().join("");
}

// Returns true for a palindrome, false otherwise.
function palindrome(string) {
  return string === reverse(string);
}
```

Reloading **palindrome.js** lets us check the effect of the **palindrome** function directly:

```
> .load palindrome.js
> palindrome("To be or not to be");
false
> palindrome("Racecar");
false
> palindrome("level");
true
```

It works!

There's one minor refinement it would be nice to add right away, which is the ability to detect palindromes independent of case. In other words, we'd like to return **true** for something like "Racecar", even though the initial "R" is capitalized. We can do this by converting the string to lowercase before making the comparison, which we can do using the **toLowerCase** method from [Section 2.5](#):

```
> let processedContent = string.toLowerCase();
> processedContent;
'racecar'
> processedContent === reverse(processedContent);
true
```

Putting this into **palindrome.js** gives [Listing 5.10](#).

Listing 5.10: Detecting palindromes independent of case.
palindrome.js

```
// Reverses a string.
function reverse(string) {
  return string.split("").reverse().join("");
}

// Returns true for a palindrome, false otherwise.
function palindrome(string) {
  let processedContent = string.toLowerCase();
  return processedContent === reverse(processedContent);
}
```

Using the REPL, we can confirm that it worked:

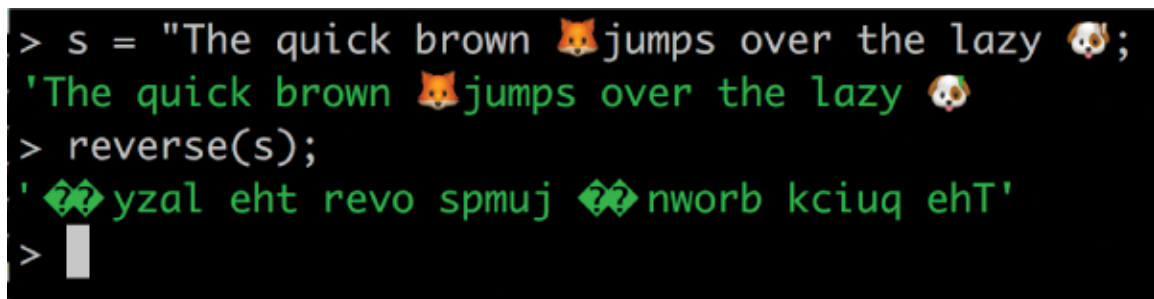
```

> .load palindrome.js
> palindrome("racecar");
true
> palindrome("Racecar");
true
> palindrome("Able was I ere I saw Elba");
true

```

5.3.1 Caveat Emoji

There's one minor caveat to the **reverse** method developed in [Listing 5.10](#), which is that it won't quite work with text that includes more complicated characters like emojis. For example, attempting to reverse a sentence containing the “fox face” and “dog face” emojis yields a garbled result, as shown in [Figure 5.7](#).



```

> s = "The quick brown 🦊 jumps over the lazy 🐶;
'The quick brown 🦊 jumps over the lazy 🐶
> reverse(s);
'🦊? yzal eht revo spmuj 🦊? nworb kciuq ehT'
>

```

Figure 5.7: A failed emoji reversal.

The reason for this is that each emoji is effectively represented as two separate characters, and splitting on the empty string as in [Listing 5.10](#) splits each emoji in half (ouch!). The solution is to create an array from a string a different way, using a custom array method called **Array.from()**:

```

> Array.from('honey badger');
[ 'h', 'o', 'n', 'e', 'y', ' ', 'b', 'a', 'd', 'g', 'e', 'r' ]

```

Replacing **split** in [Listing 5.10](#) with this improved method gives the updated **reverse** code in [Listing 5.11](#).

Listing 5.11: Improving **reverse** using **Array.from**.

```

// Reverses a string.
function reverse(string) {

```

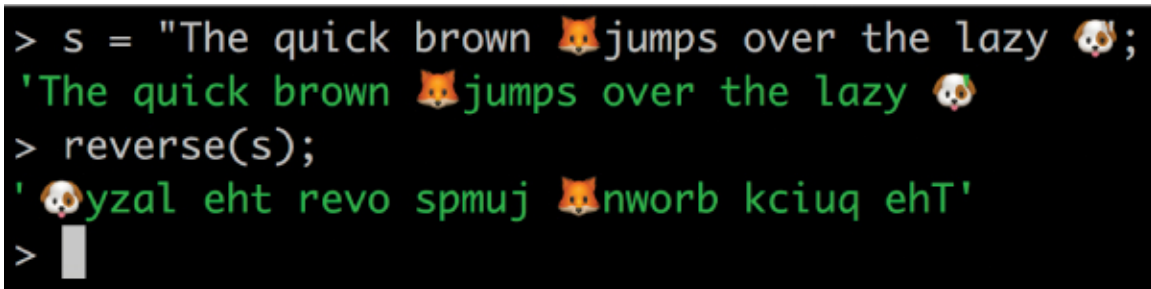
```
    return Array.from(string).reverse().join('')
  }

  // Returns true for a palindrome, false otherwise.
  function palindrome(string) {
    let processedContent = string.toLowerCase();
    return processedContent === reverse(processedContent);
  }
```

Confirming that this code works is left as an exercise.

5.3.2 Exercises

1. Using method chaining and the template in [Listing 5.12](#), write a function **emailParts** to return an array of the username and domain for a standard email address of the form **username@example.com**. *Note:* Make sure your function returns the same result for **USERNAME@EXAMPLE.COM**.
2. Using the Node REPL, confirm that the **reverse** function defined in [Listing 5.11](#) correctly reverses a string containing emojis. (You may find Emojipedia (<https://emojipedia.org/>) links to the fox face and dog face emojis helpful.) Your result should look something like [Figure 5.8](#).



```
> s = "The quick brown 🦊 jumps over the lazy 🐶";
'The quick brown 🦊 jumps over the lazy 🐶'
> reverse(s);
'🐶 yzal eht revo spmuj 🦊 nworb kciuq ehT'
>
```

Figure 5.8: A successful emoji reversal.

Listing 5.12: Returning the parts of an email.

```
> function emailParts(email) {
  // FILL IN
}
```

5.4 Iteration for Each

So far, we’ve seen several examples of *iteration*: for strings ([Section 2.6](#)), arrays ([Section 3.5](#)), and objects ([Section 4.5](#))—all based on the **for** loop. In this section, we’ll learn how to use **forEach** loops, which iterate through each element in an array, without the inconvenience of an auxiliary index variable.

Doing an operation “for each” element in an array means we can change from this:

```
for (let i = 0; i < array.length; i++) {  
  console.log(array[i]);  
}
```

to this:

```
array.forEach(function(element) {  
  console.log(element);  
});
```

The latter code allows us to perform actions on each array element directly, without having to access it using **array[i]**.

You can see why we had to wait until now: **forEach** requires that we use a function—in particular, a nameless, or *anonymous*, function—to create a variable for each element in the array.⁴ That said, I find it helps not to pronounce “function” (whether aloud or in your head), so that it sounds like “array: for each element <do something>.”

4. A function (whether named or anonymous) with data attached in this manner is known as a *closure*.

To get a better understanding of **forEach**, let’s look at a concrete example in the REPL:

```
> [42, 17, 85].forEach(function(element) {  
  console.log(element);  
});  
42  
17  
85
```

What's going on here is that **forEach** itself takes a function as an argument, which then returns each element of the corresponding array in turn. The syntax might look a little strange, but this pattern of passing a function to a method is a common one, and you'll soon become accustomed to it. Don't worry too much about exactly what's going on under the hood—instead, focus on the concrete effects.

Using our newfound **forEach** powers, we can rewrite each of the previously encountered **for** loops using **forEach**, starting with the array iteration from [Listing 3.4](#). For convenience, we'll put the code in a file and execute it at the command line:

```
$ touch foreach.js
```

To perform the iteration, all we need is a **forEach** loop whose contents print the element itself instead of printing `a[i]`. The result is shown in [Listing 5.13](#).

Listing 5.13: Iterating through an array with a **forEach** loop.

foreach.js

```
let a = ["ant", "bat", "cat", 42];
a.forEach(function(element) {
  console.log(element);
});
```

Executing the program in [Listing 5.13](#) at the command line results in the same output we saw in [Listing 3.4](#), as shown in [Listing 5.14](#).

Listing 5.14: The output of an array iteration.

```
$ node foreach.js
ant
cat
bat
42
```

Now let's use **forEach** to rewrite the string iteration from [Listing 2.18](#). Our technique will be to create an array from the string, and then use **forEach** to iterate one element at a time. For the first step, we'll be creating an array from

the string using the **Array.from** method introduced at the end of [Section 5.3](#) ([Listing 5.11](#)):

```
> Array.from("honey badger");  
[ 'h', 'o', 'n', 'e', 'y', ' ', 'b', 'a', 'd', 'g', 'e', 'r' ]
```

The result is an array of characters, which we can then iterate through using **forEach**.

We'll start by including the **soliloquy** variable from [Listing 2.15](#) in the **foreach.js** file, and then use **Array.from** and **forEach**. The resulting code (which we'll place after the array iteration from [Listing 5.13](#)) appears as in [Listing 5.15](#).

Listing 5.15: Using a **forEach** loop to iterate through a string.
foreach.js

```
.  
.   
.   
let soliloquy = "To be, or not to be, that is the question:";  
Array.from(soliloquy).forEach(function(character) {  
    console.log(character);  
});
```

Executing the program in [Listing 5.15](#) at the command line results in the same output we saw in [Listing 2.18](#) (preceded in this case by the output from [Listing 5.14](#)), as shown in [Listing 5.16](#).

Listing 5.16: The output of a string iteration.

```
$ node foreach.js  
ant  
bat  
cat  
42  
T  
o  
  
b  
e  
.  
.  
.
```

t
i
o
n
:

Using the **forEach** method, we can iterate directly through the elements in an array, thereby avoiding having to type out Mike Vanier’s *bête noire*, “for (i = 0; i < N; i++)”. The result is cleaner code and a happier programmer (Figure 5.9).



Figure 5.9: Using **forEach** has made Mike Vanier a little happier.

5.4.1 Exercises

1. Rewrite the **forEach** loop in Listing 5.13 using the fat arrow notation from Section 5.1.2.
2. We saw in Listing 5.1 how to define a number comparison function that let us sort JavaScript arrays numerically. There we used return values of **1**, **-1**, and **0**, but it turns out **sort** only cares about the *sign* of the comparison, so **17** is the same as **1**, **-42** is the same as **-1**, etc. For numbers **a** and **b**, the value **a - b** has the right sign, so show that the code in Listing 5.17, which uses an anonymous function, correctly sorts the array.
3. Write a **forEach** loop to print the values of the previous exercise.

Listing 5.17: Sorting an array the anonymous way.

```
> let a = [8, 17, 42, 99];  
> a.sort(function(a, b) { return a - b; });  
[ 8, 17, 42, 99 ]
```
